INTRODUCTION TO ALGORITHMS

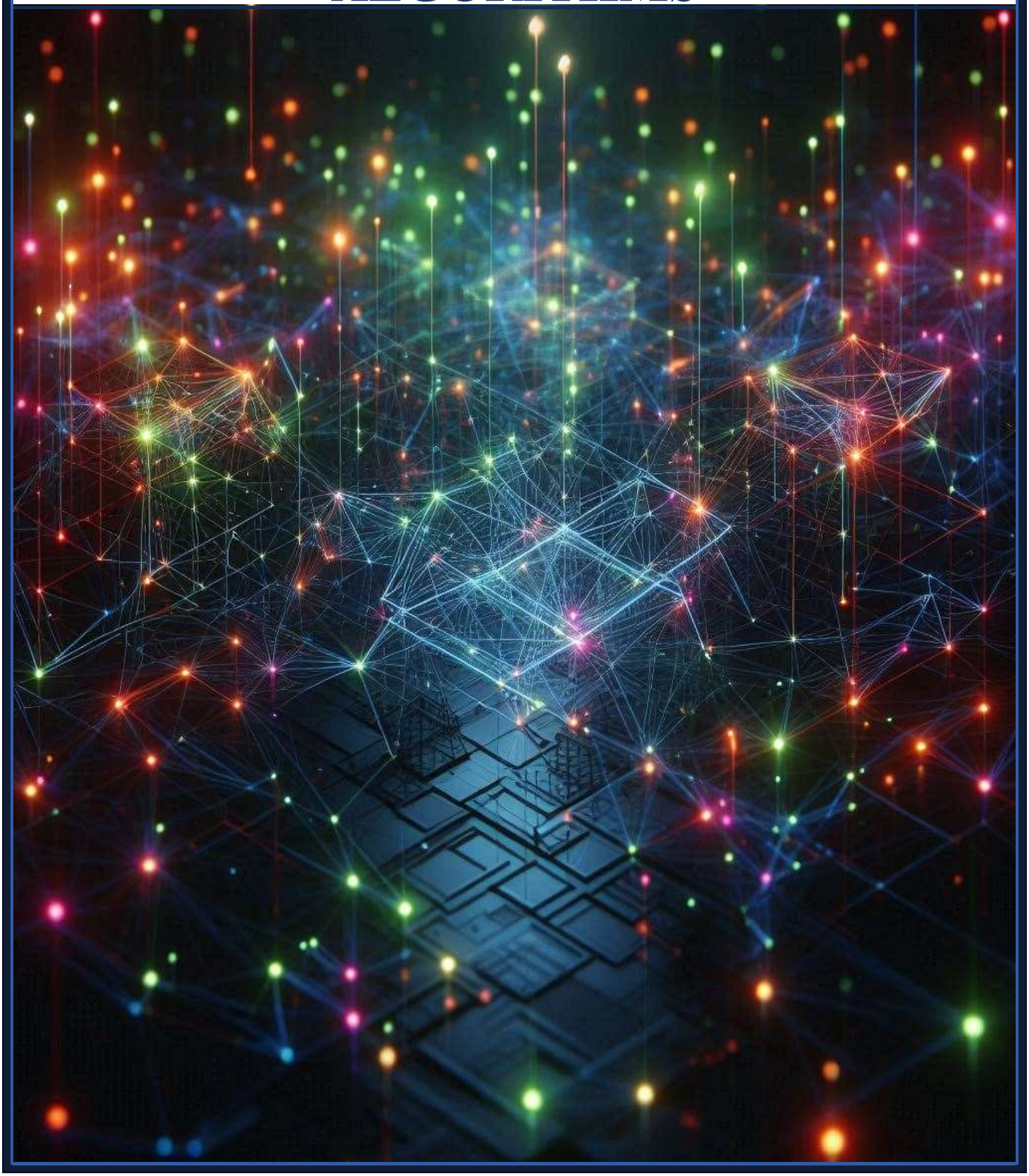


Table of Contents

MODULE 01

INTRODUCTION TO ALGORITHMS

1. BASIC NOTIONS OF AN ALGORITHM

Algorithms are the cornerstone of computer science, providing a step-by-step procedure to solve a given problem. To make it clear, **an algorithm as a finite set of well-defined instructions for solving a specific problem or performing a task.** Think of it as a recipe that tells a computer how to transform input data into the desired output. They form the foundation for various software applications and systems. Here, are the fundamental concepts of algorithms:

- **Precision:** Each step in an algorithm should be clearly defined and **unambiguous (not open to more than one interpretation)**. There should be no room for interpretation or ambiguity.
- **Finiteness:** An algorithm should terminate after a finite number of steps. It should not have an infinite loop or an indefinite process.
- **Input:** An algorithm takes input data, which can be of various types (numbers, characters, arrays, etc.). The input defines the problem to be solved.
- **Output:** An algorithm produces an output, which is the solution to the given problem. The output can be of various forms, depending on the nature of the problem.
- **Effectiveness:** An algorithm should be effective in solving the intended problem. It should produce the correct output for all valid inputs.
- **Generality:** An algorithm should be applicable to a set of problems rather than a single specific problem. It should be general enough to solve all problems of a particular type.
- **Efficiency:** An algorithm should be efficient in terms of time and space complexity. This means it should execute quickly and use minimal memory resources.
- **Correctness:** An algorithm should be correct in the sense that it produces the desired output for all valid inputs.
- **Clarity:** An algorithm should be easy to understand and follow. Clear and concise notation is essential for readability.
- **Scalability:** A scalable algorithm can handle increasing amounts of data or more complex problems without a significant drop in performance.

Example 1: Consider a simple algorithm for adding two numbers

- **Input:** Two numbers, say (a) and (b).
- **Process:** Add the two numbers.
- **Output:** The sum of (a) and (b).

This algorithm is finite (it ends after the addition), definite (the steps are clear), has input and output, is effective (simple addition), general (works for any two numbers), correct (produces the correct sum), efficient (addition is a basic operation), and scalable (can handle any size of numbers).

Key Elements of Algorithms

- **Control Flow:** The sequence of steps in an algorithm, including decision-making ***(if-else statements),*** loops ***(for, while)***, and function calls.
- **Data Structures:** The organization and storage of data within an algorithm. Common data structures include ***arrays, linked lists, stacks, queues, trees, and graphs.***
- **Variables:** Variables are used to store and manipulate data within an algorithm.
- **Functions:** Functions are reusable blocks of code that can be called multiple times within an algorithm.

By understanding these fundamental concepts, you can effectively design and analyse algorithms for various problem-solving tasks.

1.1. Criteria of Classifying Algorithms

Algorithms can be classified based on various criteria, each highlighting different aspects of their design and functionality. Here are some common criteria for classifying algorithms:

Implementation Method

- **Recursive Algorithms:** These algorithms call themselves repeatedly until a base condition is met. Example: **Tower of Hanoi.**
- **Iterative Algorithms:** These use loops to repeat a set of instructions until a condition is satisfied. Example: **Factorial calculation using a loop.**

Design Method

- **Greedy Algorithms:** Make the best choice at each step with the hope of finding the global optimum. Example: **Dijkstra's algorithm for shortest paths.**
- **Divide and Conquer:** Break the problem into smaller subproblems, solve them recursively, and combine their solutions. Example: **Merge Sort.**
- **Dynamic Programming:** Solve complex problems by breaking them down into simpler subproblems and storing the results of subproblems to avoid redundant computations. Example: **Fibonacci sequence calculation.**

Purpose

- **Sorting Algorithms:** Arrange data in a particular order. Example: **QuickSort, BubbleSort.**
- **Search Algorithms:** Find specific elements within a dataset. Example: **Binary Search, Linear Search.**

Operational Structure

- **Serial Algorithms:** Execute one instruction at a time. Example: **Traditional sorting algorithms.**
- **Parallel Algorithms:** Divide the problem into subproblems that can be solved simultaneously on multiple processors. Example: Parallel versions of **QuickSort.**
- **Distributed Algorithms:** Solve problems across multiple machines or processors. Example: **MapReduce.**

Deterministic vs. Non-Deterministic

- **Deterministic Algorithms:** Produce the same output for a given input every time. Example: **Euclidean algorithm for GCD.**
- **Non-Deterministic Algorithms:** May produce different outputs for the same input on different runs, often using randomness. Example: **Monte Carlo algorithms.**

Exact vs. Approximate

- **Exact Algorithms:** Always produce the optimal solution. Example: **Binary Search.**
- **Approximate Algorithms:** Provide solutions that are close to the optimal, often used when exact solutions are computationally expensive. Example: **Approximation algorithms for NP-hard problems.**

Complexity

- **Time Complexity:** Measure of the time an algorithm takes to complete as a function of the input size. Example: **O(n log n) for Merge Sort.**
- **Space Complexity:** Measure of the memory an algorithm uses as a function of the input size. Example: **O(1) for in-place sorting algorithms.**

Application Domain

- **Cryptographic Algorithms:** Used for securing data. Example: **RSA, AES.**
- **Machine Learning Algorithms:** Used for data analysis and predictive modelling. Example: **Decision Trees, Neural Networks.**

Understanding these criteria helps in selecting the right algorithm for a specific problem, ensuring efficiency and effectiveness in solving computational tasks

1.2. How to measure the complexity of an Algorithm

Algorithm complexity is a measure of how efficient an algorithm is in terms of time and space resources it consumes. Measuring the complexity of an algorithm involves analysing how the algorithm's resource usage grows as the size of the input increases.

Understanding the complexity of an algorithm is crucial for predicting its performance and making informed decisions about its suitability for different applications. The two main types of complexity are **time complexity** and **space complexity.** Here's a detailed explanation of each:

a. Time Complexity

Measures the number of operations an algorithm performs as a function of the input size.

Common notations:

- **O(n):** Linear time complexity, meaning the algorithm's running time grows linearly with the input size.
- **O(n^2):** Quadratic time complexity, meaning the running time grows quadratically with the input size.
- **O(log n):** Logarithmic time complexity, meaning the running time grows logarithmically with the input size.

- **O(n log n):** Linearithmic time complexity, a combination of linear and logarithmic growth.
- **O(1):** Constant time complexity, meaning the running time is independent of the input size.
- **O(2^n):** Exponential time. The running time doubles with each additional input element.
- **O(n!):** Factorial time. The running time grows factorially with the input size.

b. Space Complexity:

Measures the amount of memory an algorithm uses as a function of the input size.

- **Common notations:** Similar to time complexity notations.

Factors Affecting Complexity:

- **Input size:** The larger the input, the more time and space an algorithm may require.
- **Algorithm design:** The choice of algorithm and its implementation can significantly affect complexity.
- **Data structures:** The use of appropriate data structures can optimize performance.

Examples:

- **Linear Search:** O(n) time complexity, as it needs to examine each element in the worst case.
- **Binary Search:** O(log n) time complexity, as it repeatedly divides the search space in half.
- **Bubble Sort:** O(n^2) time complexity in the worst case, but can be O(n) in the best case.
- **Merge Sort:** O(n log n) time complexity in all cases.

Best Practices for Analysing Complexity:

- **Identify the dominant operation:** Focus on the operation that is executed the most frequently.
- **Express the running time in terms of the input size:** Use the appropriate notation **(e.g., O(n), O(n^2))** to represent the relationship between the running time and the input size.
- **Consider worst-case, average-case, and best-case scenarios:** Analyse the algorithm's performance under different conditions.

By understanding algorithm complexity, you can make informed decisions about choosing the right algorithms for your applications and optimize their performance. Understanding and measuring algorithm complexity helps in selecting the most efficient algorithm for a given problem, ensuring optimal performance and resource usage

2. CONCEPTS OF ALGORITHM

The goal of algorithms is for the resolution of problems. The computer resolution of a problem generally passes through five (05) major stages:

2.1. Analysis of the Problem to be Solved

i. Understanding the Problem

- **Identify the Problem:** Clearly state what needs to be solved.
- **Determine the Input and Output:** Define what inputs are required and what outputs are expected.
- **Constraints and Requirements:** Identify any constraints (e.g., time limits, memory limits) and specific requirements.

Breaking Down the Problem

- **Decompose the Problem:** Break the problem into smaller, manageable subproblems.
- **Identify Key Operations:** Determine the fundamental operations needed to solve the problem.

Choosing the Right Approach

- **Algorithm Design Techniques:** Choose an appropriate design technique (e.g., divide and conquer, dynamic programming, greedy algorithms).
- **Data Structures:** Select suitable data structures that will efficiently support the algorithm.

Analysing the Algorithm

- **Time Complexity:** Measure how the running time of the algorithm increases with the size of the input. Use Big O notation to express this.
- **Space Complexity:** Measure the amount of memory the algorithm uses as the input size increases.

ii. Development of Algorithm

Algorithm Design

- **Top-down approach:** Break down the problem into smaller subproblems.
- **Bottom-up approach:** Start with smaller, simpler subproblems and build up to the solution.
- **Divide-and-conquer:** Divide the problem into smaller, similar subproblems, solve them recursively, and combine the solutions.
- **Dynamic programming:** Store solutions to subproblems to avoid redundant calculations.
- **Greedy algorithms:** Make locally optimal choices at each step, hoping to achieve a globally optimal solution.
- **Backtracking:** Explore all possible solutions and backtrack when a dead-end is reached.

Pseudocode

- Write a simplified, informal description of the algorithm using a combination of natural language and programming-like constructs.

- This helps in understanding the logic and structure of the algorithm before implementing it in a specific programming language.

iii. Programming

Implementation

- Translate the pseudocode into a specific programming language.
- Consider factors like efficiency, readability, and maintainability.

iv. Compilation

Compilation is the process of translating **high-level programming code** into **low-level machine code** that a computer can directly execute. It involves a series of steps that transform human-readable source code into machine-readable object code.

Key Steps in Compilation

- **Lexical Analysis:** Breaks down the source code into tokens, which are the smallest meaningful units of a program (e.g., keywords, identifiers, operators, literals).
- **Syntax Analysis:** Checks the syntax of the code to ensure it adheres to the grammar rules of the programming language. Constructs a parse tree to represent the structure of the code.
- **Semantic Analysis:** Analyses the meaning of the code, ensuring that variables are declared correctly, types are compatible, and expressions are valid. Builds a symbol table to store information about variables, functions, and other language elements.
- **Intermediate Code Generation:** Generates an intermediate representation of the code, which is often a language-independent form that is easier to optimize.
- **Code Optimization:** Applies various optimization techniques to improve the efficiency of the generated code. This can involve techniques like loop unrolling, constant folding, and dead code elimination.
- **Target Code Generation:** Translates the intermediate code into machine code specific to the target architecture **(e.g., x86, ARM).** This involves selecting appropriate instructions and addressing modes.
- **Linking:** Combines the object code with other necessary libraries or modules to create an executable program. Resolves external references and dependencies.

v. Execution

The execution phase in an algorithm program is the stage where the instructions defined by the algorithm are carried out by the computer. It involves the following steps:

- **Loading the program:** The compiled program is loaded into the computer's memory.
- **Initializing variables:** The program's variables are assigned their initial values.
- **Executing instructions:** The computer follows the sequence of instructions defined in the algorithm.
  - **Control flow statements:** The computer executes instructions based on conditional statements **(if-else)** and loops **(for, while).**
  - **Function calls:** If the algorithm involves functions, the computer jumps to the function's code, executes it, and returns to the calling point.

- **Data manipulation**: The computer performs operations on data, such as arithmetic calculations, comparisons, and assignments.
- **Output:** The program generates the desired output, which may be displayed on the screen, written to a file, or used for further processing.

Key points to remember about the execution phase

- The execution of an algorithm is determined by its control flow statements.
- Functions can be used to modularize code and improve readability.
- Data manipulation is a crucial part of the execution phase, involving operations on variables and data structures.
- The output of an algorithm depends on the specific instructions and input data.

By understanding the execution phase, you can better analyse and optimize the performance of your algorithms.

2.2. Qualities of a Good Algorithm

An algorithm must have the following qualities:

- Each action is defined in a precise way and without ambiguity.
- Each action is effective that is to say it can actually be carried out.
- What are the data, the algorithm must finish and yield results.
- It has to give the same result each time it is executed on the same set of data.
- It must consume fewer resources (time and the memory capacity) possible. This implies a good analysis of complexity in time and space of the written algorithms.

2.3. Advantages of the Algorithmic Steps

Breaking down a problem into smaller, manageable steps using algorithms offers several advantages:

- **Clarity and Understanding:** Algorithms provide a structured approach, making it easier to understand the problem and its solution. Each step is clearly defined, reducing ambiguity and confusion.
- **Efficiency:** By breaking down complex problems into smaller, more manageable steps, algorithms can often lead to more efficient solutions. Identifying patterns and common subproblems allows for optimization.
- **Modularity:** Algorithms can be broken down into smaller, reusable components (functions or procedures), promoting modularity and code reusability.
- **Problem Solving:** Algorithms provide a systematic approach to problem-solving, guiding you through the process of finding a solution. They help you identify potential solutions and evaluate their effectiveness.
- **Communication:** Algorithms can be used to communicate the solution to others, ensuring that everyone understands the process and can implement it. They serve as a blueprint for developers to follow.
- **Debugging and Testing:** Algorithms make it easier to identify and fix errors in code. By following the steps of an algorithm, you can systematically test and debug your implementation.

- **Optimization:** Analysing the steps of an algorithm can help you identify areas for optimization, such as reducing redundant calculations or improving efficiency.
- **Adaptability:** Algorithms can be adapted to solve different variations of the same problem or similar problems.
- **Automation:** Once an algorithm is implemented, it can be automated to perform tasks repeatedly, saving time and effort.

3. VARIABLES, ACTIONS & OPERATIONS

Variables in programming are like containers that hold/store data values like numbers or characters. They are used to store data that can change during the execution of a program. Think of them as labels or names that refer to specific data.

Key Characteristics of Variables

- **Name:** Each variable has a unique name that you use to refer to it.
- **Type:** Variables have a type that determines the kind of data they can hold (e.g., integer, float, character, string).
- **Value:** The current data stored in the variable. This value can be changed during program execution.

Examples of Variables

- age **(integer)**
- name **(string)**
- temperature **(float)**
- is_raining **(boolean)**

Declaring Variables

Before using a variable, you typically need to declare it. This involves specifying its **name** and **type**. The syntax for declaring variables varies depending on the programming language. For example, in C, you might declare a variable like this:

```
int age; // Declares an integer variable named 'age'
```

Assigning Values

Once a variable is declared, you can assign a value to it using the assignment operator **(=).** For example:

```
age = 25; // Assigns the value 25 to the variable 'age'
```

Using Variables

Variables can be used in various ways within a program, such as:

- Performing calculations
- Making decisions (e.g., in if-else statements)
- Storing and retrieving data.

Note: In many other programming languages (like Python, Java, and C++), you would normally use a print function to display the value of a variable. However, this is not possible in C:

Example 2: How to make use of a variable when writing a C program

```
#include <stdio.h>
int main() {
  int myNum = 15;
  printf(myNum); // Nothing happens
  return 0;
}
```

To output variables in C, you must get familiar with something called **"format specifiers".**

Format Specifiers

Format specifiers are used together with the **printf()** function to tell the compiler what type of data the variable is storing. It is basically a placeholder for the variable value. A format specifier starts with a percentage sign **%,** followed by **a character**. For example, to output the value of an **int variable**, use the format specifier **%d** surrounded by **double quotes ("")**, inside the **printf() function:**

Example 3: How to make use of a format specifier when writing a C program

```
#include <stdio.h>
int main() {
  int myNum = 15;
  printf("%d", myNum);
  return 0;
}
```

You can also just print a value without storing it in a variable, as long as you use the correct format specifier:

3.1. Identifiers

Identifiers in an algorithm are names given to entities like **variables**, **functions**, **classes**, or other program elements. They serve as labels or tags that allow you to refer to and manipulate these elements within your code.

Key Characteristics of Identifiers

- **Unique:** Each identifier within a given scope (e.g., a function or class) must be unique.
- **Meaningful:** Identifiers should be chosen to reflect their purpose, making the code more readable and understandable.

- **Syntax:** The syntax for identifiers varies depending on the programming language. Generally, they can contain letters, numbers, and underscores, but may have restrictions on the first character.

Examples of Identifiers

- age (variable)
- calculateArea (function)
- Person (class)
- my_variable (variable)

Best Practices for Identifiers

- Use meaningful names that reflect the purpose of the identifier.
- Avoid using reserved words or keywords from the programming language.
- Use consistent naming conventions within your code.
- Consider using **camelCase** or **PascalCase** for variable and function names.

3.2. DataTypes

Data types define the kind of data that a variable can hold in an algorithm. They determine the range of values a variable can store, the operations that can be performed on it, and how the data is represented in memory.

Common Data Types

- **Numeric:**
  - **Integer:** Whole numbers (e.g., int, long).
  - **Floating-point:** Real numbers with decimal points (e.g., float, double).
- **Character:** Single characters (e.g., char).
- **String:** Sequences of characters (e.g., string).
- **Boolean:** True or false values (e.g., bool).
- **Arrays:** Collections of elements of the same data type (e.g., int[], string[]).
- **Enums:** Enumerated types that define a set of named constants (e.g., enum Color { RED, GREEN, BLUE }).

Importance of Data Types

- **Memory Allocation:** The data type determines the amount of memory needed to store the value.
- **Valid Operations:** Different data types support different operations (e.g., arithmetic operations on numbers, string concatenation).
- **Type Safety:** Using correct data types helps prevent errors and ensures that operations are performed on compatible data.
- **Readability:** Explicitly declaring data types improves code readability and maintainability.

Example 4: A C program that highlights different data tpes

```
#include <stdio.h>
int main() {
  // Create variables
  int myNum = 5;                  // Integer (whole number)
  float myFloatNum = 5.99;    // Floating point number
  char myLetter = 'D';          // Character

// Print variables
  printf("%d\n", myNum);
  printf("%f\n", myFloatNum);
  printf("%c\n", myLetter);
  return 0;
}
```

Exercise 1: Write an algorithm that displays on the terminal **"My favourite number is"** followed by the value stored in your variable created**.** To combine both text and a variable, separate them with a comma **(,)** inside the **printf()** function.

Exercise 2: Write an algorithm that displays **"My number is"** the value stored in your variable and **"My letter is"** followed by the value stored in your variable. With aim of printing different types in a single **printf() function.**

3.3. Operators

Operators in an algorithm are **symbols** or **keywords** used to perform specific operations on data. They are essential for manipulating values and controlling the flow of execution within a program. The most common types of operators include;

Arithmetic Operators

- Addition (+)
- Subtraction (-)
- Multiplication (*)
- Division (/)
- Modulus (%) - **Returns the remainder of division**

Relational Operators

- Equal to (==)
- Not equal to (!=)
- Greater than (>)
- Less than (<)
- Greater than or equal to (>=)
- Less than or equal to (<=)

Logical Operators

- Logical AND (&&)
- Logical OR (||)
- Logical NOT (!)

Assignment Operators

- Assignment (=)
- Addition assignment (+=)
- Subtraction assignment (-=)
- Multiplication assignment (*=)
- Division assignment (/=)
- Modulus assignment (%=)

Bitwise Operators

- Bitwise AND (&)
- Bitwise OR (|)
- Bitwise XOR (^)
- Bitwise NOT (~)
- Left shift (<<)
- Right shift (>>)

Example 5: A C program that demonstrates how each operator in used

```
int x = 10;
int y = 5;

// Arithmetic operators
int sum = x + y; // 15
int difference = x - y; // 5
int product = x * y; // 50
int quotient = x / y; // 2
int remainder = x % y; // 0

// Relational operators
bool isEqual = (x == y); // false
bool isNotEqual = (x != y); // true
bool isGreater = (x > y); // true

// Logical operators
bool andResult = (x > 0) && (y > 0); // true
bool orResult = (x == 0) || (y == 0); // false

// Assignment operators
x += 2; // x becomes 12
y -= 3; // y becomes 2
```

Exercise 3: Complete the above algorithm in order to display the expected output as shown in the comments

3.3.1. Concatenation Operator

In programming**, concatenation operators are used to combine or join two or more strings or other data types end-to-end into a single value.** They are essential for building complex strings and manipulating text data. In C, this is typically done using the **strcat()** function from the **<string.h>** library.

Example 6: A program that demonstrates the concept of concatenation

```
#include <stdio.h>
#include <string.h>

int main() {
    char str1[50] = "Hello";
    char str2[50] = "World!";
    char result[100];

    strcpy(result, str1);  // Copy str1 into result
    strcat(result, " ");   // Add a space to result
    strcat(result, str2);  // Concatenate str2 to result

    printf("%s", result);  // Print the concatenated string
    return 0;
}
```

4. CONTROL STRUCTURES

Control structures are essential components of algorithms that determine the flow of execution. They allow you to make decisions, repeat code blocks, and control the order in which instructions are executed. There exist different types of control structures, as stated below;

- **Sequential Control:** Being the most basic form of control flow, statements are executed in a **linear order,** that is one after another in the order they appear.

Example 7: A C program that sums 2 numbers following a linear order

```
#include <stdio.h>
int main() {
    int a = 5;
    int b = 10;
    int sum = a + b;
    printf("Sum: %d\n", sum);  // Output: Sum: 15
    return 0;
}
```

- **Selection Control (Conditional Statements):** These structures allow the program to choose different paths of execution based on conditions. Several different types exist, as stated below;

If-Else Statement

```
if (condition) {
   // Code to execute if condition is true
} else {
   // Code to execute if condition is false
}
```

Example 8: A C program demonstrating an If-Else Statement

```
#include <stdio.h>
int main() {

   int age;
   printf("Enter your age: ");
   scanf("%d", &age);

   if (age >= 18) {
      printf("You are an adult.\n");
   } else {
      printf("You are a minor.\n");
   }
  return 0;
}
```

This program prompts the user to enter their age, then uses an **if-else** statement to determine whether they are an adult or a minor. The **if condition** checks if the age is **greater than** or **equal to 18**. If true, the message **"You are an adult."** is printed. Otherwise, the message **"You are a minor."** is printed.

Exercise 4: Write a C program that uses the if-else statement to determine whether a number entered by the user is even or odd.

Else-If Statement

```
if (condition1) {
   // Code to execute if condition 1 is true
} else if (condition2) {
   // Code to execute if condition 2 is true and condition 1 is false
} else {
   // Code to execute if all conditions are false
}
```

Example 9: A C program demonstrating an Else-If Statement

```
#include <stdio.h>
int main() {

   int number;
   printf("Enter a number: ");
   scanf("%d", &number);

   if (number > 10) {
      printf("The number is positive.\n");
   } else if (number < 5) {
      printf("The number is negative.\n");
   } else {
      printf("The number is zero.\n");
   }
  return 0;
}
```

Exercise 5: Write a C program making use of else-if statement to determine a student's grade based on their score. Whereby a grade of **>=90** gives **Excellent**, **>= 80** gives **Very Good, >=70** gives **Good** and a **>=50** gives a **Passed** mark.

Switch

```
switch (expression) {
   case value1:
      // Code to execute if expression equals value 1
      break;
   case value2:
      // Code to execute if expression equals value 2
      break;
   default:
      // Code to execute if expression doesn't match any case
}
```

Example 10: A C program demonstrating an Switch Statement

```
#include <stdio.h>
int main() {

   int choice;
   printf("Menu:\n");
   printf("1. Add\n");
   printf("2. Subtract\n");
   printf("3. Multiply\n");
   printf("4. Divide\n");
   printf("Enter your choice: ");
   scanf("%d", &choice);
```

```
    int num1, num2;
    printf("Enter your first number: ");
    scanf("%d", &num1);
    printf("Enter your second number: ");
    scanf("%d", &num2);

    switch (choice) {
        case 1:
            printf("Result: %d\n", num1 + num2);
            break;
        case 2:
            printf("Result: %d\n", num1 - num2);
            break;
        case 3:
            printf("Result: %d\n", num1 * num2);
            break;
        case 4:
            if (num2 == 0) {
                printf("Error: Division by zero is not allowed.\n");
            } else {
                printf("Result: %.2f\n", (float)num1 / num2);
            }
            break;
        default:
            printf("Invalid choice.\n");
    }
    return 0;
}
```

This C program demonstrates the use of a switch statement to create a simple menu-driven calculator. The user is prompted to enter a choice for the desired operation, and the **switch statement** is used to execute the corresponding code based on the user's input.

5. ITERATIONS AND LOOPS

Iteration refers to the repetition of a block of code multiple times. It's a fundamental control flow structure used in algorithms to perform tasks repeatedly until a certain condition is met. Meanwhile, loops are the programming constructs that implement iteration. Several types of loops exist, as seen below;

i. **For Loop:** It is used when the number of iterations is known in advance.

Syntax:

```
for (initialization; condition; increment) {
    // Code to be executed
}
```

Example 11: A C program that uses a for loop to print numbers from 1 to 10

```
#include <stdio.h>
int main() {
   // For loop to print numbers from 1 to 10
   for (int i = 1; i <= 10; i++) {
      printf("%d\n", i);
   }
   return 0;
}
```

Explanation:

- **Initialization: int i = 1**; initializes the loop control variable **i** to **1.**
- **Condition: i <= 10;** checks if **i** is less than or equal to **10.** If true, the loop continues; otherwise, it stops.
- **Increment: i++** increments the value of **i by 1** after each iteration.
- **Loop Body: printf("%d\n", i);** prints the current value of **i.**

ii. **While Loop:** It is used when the number of iterations is not known beforehand. The loop continues as long as a specified condition is true.

Syntax:

```
while (condition) {
   // Code to execute repeatedly
}
```

Example 12: A C program that uses a do-while loop to print numbers from 1 to 10

```
#include <stdio.h>
int main() {
   int i = 1;  // Initialization

   // While loop to print numbers from 1 to 10
   while (i <= 10) {
      printf("%d\n", i);
      i++;  // Increment
   }
  return 0;
}
```

Explanation:

- **Initialization: int i = 1;** initializes the loop control variable **i** to **1.**
- **Condition: i <= 10;** checks if **i** is less than or equal to **10.** If true, the loop continues; otherwise, it stops.
- **Loop Body: printf("%d\n", i);** prints the current value of **i.**

- **Increment: i++;** increments the value of **i** by **1** after each iteration.

iii. **Do-While Loop:** Though similar to the while loop, the condition is checked after the loop body is executed at least once.

Syntax

```
do {
   // Code to be executed
} while (condition);
```

Example 13: A C program that uses a do-while loop to print numbers from 1 to 10

```
#include <stdio.h>
 int main() {
   int i = 1;  // Initialization

   // Do-while loop to print numbers from 1 to 10
   do {
      printf("%d\n", i);
      i++;  // Increment
   } while (i <= 10);  // Condition
   return 0;
}
```

Explanation

- **Initialization: int i = 1;** initializes the loop control variable **i** to **1.**
- **Loop Body: printf("%d\n", i);** prints the current value of **i.**
- **Increment: i++;** increments the value of **i** by **1** after each iteration.
- **Condition: while (i <= 10);** checks if **i** is less than or equal to **10.** If true, the loop continues; otherwise, it stops.

The key difference between a **do-while loop** and a **while loop** is that the do-while loop guarantees that the loop body will be executed at least once, even if the condition is false initially.

Key Points

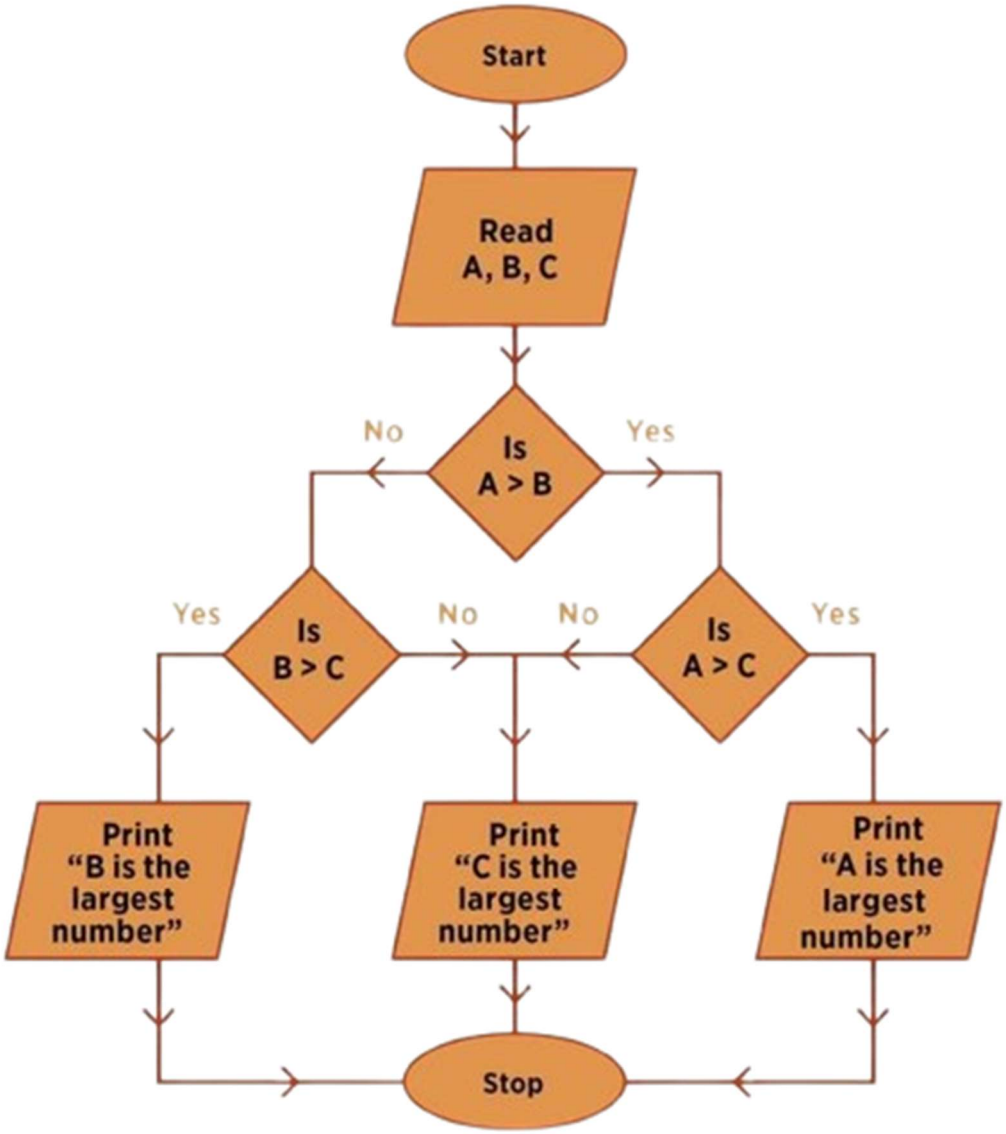
- Loops are essential for repetitive tasks and iterative algorithms.
- The choice of loop depends on the specific requirements of the algorithm.
- It's important to avoid infinite loops, which can lead to program crashes.
- Nested loops can be used to perform multiple levels of iteration.

By understanding iteration and loops, you can effectively control the flow of execution in your algorithms and perform repetitive tasks efficiently.

6. FLOWCHARTS

A flowchart is a visual diagram that represents the sequence of steps in a process, system, or algorithm. It uses standardized symbols connected by arrows to show the flow of control or data, making complex procedures easier to understand, analyse, and communicate. Flowcharts are widely used in programming, business processes, education, and engineering

Or in simpler terms, **a flowchart is the diagrammatic representation of an algorithm,** being a step-by-step approach to solving a task. As shown in the diagram below.



6.1. Components of a Flowchart Diagram

A flowchart's components include standardized geometric symbols for different process elements, such as **ovals for start/end points, rectangles for tasks, diamonds for decisions, and parallelograms for inputs/outputs.** These symbols are connected by **arrows, which indicate the direction and flow of the process from one step to the next.** As shown below.

Symbol	Name	Function
	Start/end	An oval represents a start or end point
	Arrows	A line is a connector that shows relationships between the representative shapes
	Input/Output	A parallelogram represents input or output
	Process	A rectangle represents a process
	Decision	A diamond indicates a decision

6.1.1. Additional Elements for Complexity

- **Document Symbol:** Represents a document used or created in the process.
- **Data Symbol:** Denotes the presence of data.
- **Manual Input Symbol:** Shows a step where information is provided manually by a user.
- **Connector (On-page or Off-page):** Used to connect different parts of a flowchart, especially when a diagram is too large for a single page.
- **Annotation:** A comment or brief explanation added to a symbol or a portion of the flowchart.

6.2. FROM FLOWCHART TO PSEUDOCODE

A pseudocode is a simplified, human-readable way of describing the logic of a program or algorithm without using strict programming syntax. It's like writing code in plain English (or French, or any language you understand), making it easier to plan, teach, or explain how a program works.

Question 1: What are the benefits of using a Pseudocode?

- **Clarity:** Helps learners understand logic before diving into syntax.
- **Planning:** Useful for designing algorithms before coding.
- **Teaching Tool:** Bridges the gap between concept and implementation.
- **Language-Neutral:** Can be adapted to any programming language **(C#, Python, VB.NET, and more.).**

Example 14: Let's say we want to write a program that checks if a student passed based on their score.

```
START
  DISPLAY "Enter a student's score: "
  INPUT score

  IF score >= 50 THEN
    DISPLAY "Student passed."
  ELSE
    DISPLAY "Student failed."
  ENDIF
END
```

Breakdown of Components

KEYWORD	PURPOSE
START / END	Marks the beginning and end of the algorithm
DISPLAY	Outputs a message to the user
INPUT	Receives user input
IF...THEN...ELSE	Decision-making logic
ENDIF	Ends the conditional block

Exercise 6: Convert the existing flowchart above to its equivalent pseudocode and write its existing C program.

Exercise 7: From the pseudocode stated above write its existing C program.

7. ARRAYS AND SEARCH ALGORITHMS

Arrays are a fundamental data structure in programming, representing a collection of elements of the same data type. They are often used to store and manipulate multiple related values efficiently. Key characteristics of an array include;

- **Elements:** The individual values stored in an array are called elements.
- **Index:** Each element is associated with an index, which is used to access the element.
- **Data Type:** All elements in an array must be of the same data type **(e.g., int, float, char).**

Declaration and Initialization

In C, arrays are declared using the following syntax:

```
data_type array_name[size];
```

Where

- **data_type** specifies the type of elements in the array.
- **array_name** is a unique identifier for the array.

- **size** is the number of elements the array can hold.

Example 15: Declares an array of 5 integers and initializes the array with values.

```
int numbers[5]; // Declares an array of 5 integers
int numbers[5] = {10, 20, 30, 40, 50}; // Initializing an array with values
```

Accessing Elements

Elements in an array are accessed using their index, which starts from 0.

Syntax

```
int firstElement = numbers[0]; // Accesses the first element
```

Operations on Arrays

- **Traversing:** Iterating through all elements of an array.
- **Searching:** Finding a specific element in an array **(e.g., linear search, binary search).**
- **Sorting:** Arranging elements in a specific order **(e.g., bubble sort, merge sort, quick sort).**
- **Insertion:** Adding a new element to an array.
- **Deletion:** Removing an element from an array.

Example 16: A C that demonstrates the different operations performed in an array

```
#include <stdio.h>
int main() {
    int numbers[5] = {10, 20, 30, 40, 50};
    int size = sizeof(numbers) / sizeof(numbers[0]);

    // Traversing the array
    for (int i = 0; i < size; i++) {
        printf("%d ", numbers[i]);
    }
    printf("\n");

    // Searching for an element
    int key = 30;
    int index = -1;
    for (int i = 0; i < size; i++) {
        if (numbers[i] == key) {
            index = i;
            break;
        }
    }
    if (index != -1) {
        printf("Element found at index %d\n", index);
    } else {
```

```
        printf("Element not found\n");
    }
    return 0;
}
```

Explanation

```
#include <stdio.h>
```

- This line includes the standard input-output library, which is necessary for using printf and other standard I/O functions.

```
int main() {
```

- This is the main function where the execution of the program begins.

```
    int numbers[5] = {10, 20, 30, 40, 50};
```

- Declares an array number of size 5 and initializes it with the values **{10, 20, 30, 40, 50}.**

```
    int size = sizeof(numbers) / sizeof(numbers[0]);
```

- Calculates the size of the array. sizeof(numbers) gives the total memory size of the array, and sizeof(numbers[0]) gives the size of one element. Dividing these gives the number of elements in the array.

```
    // Traversing the array
    for (int i = 0; i < size; i++) {
        printf("%d ", numbers[i]);
    }
    printf("\n");
```

- This loop iterates through the array and prints each element followed by a space. After the loop, printf("\n"); prints a newline character.

```
    // Searching for an element
    int key = 30;
    int index = -1;
```

- Initializes key with the value 30, which is the element to search for in the array. index is initialized to -1 to indicate that the element has not been found yet.

```
    for (int i = 0; i < size; i++) {
        if (numbers[i] == key) {
            index = i;
            break;
```

```
    }
}
```

- This loop searches for the key in the array. If numbers[i] equals key, index is set to i (the current index), and the loop breaks.

```
if (index != -1) {
    printf("Element found at index %d\n", index);
} else {
    printf("Element not found\n");
}
```

- After the loop, this if statement checks if index is not -1. If true, it prints the index where the element was found. Otherwise, it prints "Element not found".

```
    return 0;
}
```

- Returns 0 to indicate that the program executed successfully.

This program demonstrates basic array operations: traversing an array to print its elements and searching for a specific element within the array.

Arrays are a fundamental data structure in programming, providing a convenient way to store and manipulate collections of elements. By understanding arrays and their operations, you can write efficient and effective algorithms.

7.1. SEARCHES IN AN ARRAY

There are several techniques to search for elements in an array, each with its own advantages and disadvantages depending on the specific use case and the characteristics of the array. Here are some of the most common searching algorithms:

7.1.1. Sequential Search

A sequential search algorithm, also known as a linear search, is a simple method for finding a specific element in a list or array. ***A real-life example includes you performing a search for a file by name on your computer or within a folder, the system may use linear search to scan through the directory until it finds the desired file.*** Here's a detailed explanation:

Start at the Beginning

- The algorithm begins at the first element of the list.

Compare Each Element

- It compares each element in the list with the target value (the value you are searching for).

Continue Until Found or End

- The search continues sequentially through the list until the target value is found or the end of the list is reached.

Return Result

- If the target value is found, the algorithm returns the index of the element.
- If the target value is not found, it typically returns a sentinel value (like -1) to indicate that the element is not present in the list.

Example 17: A C program demonstrating a sequential search in an array

```
#include <stdio.h>
int sequentialSearch(int arr[], int size, int target) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == target) {
            return i; // Target found, return index
        }
    }
    return -1; // Target not found, return -1
}

int main() {
    int numbers[5] = {10, 20, 30, 40, 50};
    int target = 30;
    int index = sequentialSearch(numbers, 5, target);

    if (index != -1) {
        printf("Element found at index %d\n", index);
    } else {
        printf("Element not found\n");
    }

    return 0;
}
```

Characteristics

- **Time Complexity: O(n),** where n is the number of elements in the list. This is because, in the worst case, the algorithm may need to check every element.
- **Space Complexity: O(1),** as it requires a constant amount of additional memory regardless of the size of the list.

Advantages

- **Simplicity:** Easy to understand and implement.
- **No Assumptions:** Does not require the list to be sorted.

Disadvantages

- **Inefficiency:** Can be slow for large lists because it checks each element one by one.

Sequential search is useful for small or unsorted lists where the simplicity of the algorithm outweighs its inefficiency.

7.1.2. Binary Search Algorithm

Binary Search is an efficient algorithm used to search for a specific element in a sorted array. It works by repeatedly dividing the search interval in half until the target element is found or the search interval becomes empty. ***A real-life example includes a phone book which can be implemented as a sorted array, and a binary search allows quick lookups of phone numbers based on names.*** The steps involved include;

i. **Initialize:** Set the left and right pointers to the first and last indices of the array, respectively.
ii. **Check Middle Element:** Calculate the middle index of the current search interval.
iii. **Compare:** Compare the target element with the middle element:
- If the target element is less than the middle element, search the left half of the array.
- If the target element is greater than the middle element, search the right half of the array.
- If the target element is equal to the middle element, the search is successful.

iv. **Repeat:** Repeat steps 2 and 3 until the target element is found or the search interval becomes empty.

Now, let's have a practical approach on how a Binary Search is being performed cause from here they can be implemented in two ways which are discussed below.

- Iterative Method
- Recursive Method

The recursive method follows the divide and conquer approach. The general steps for both methods are discussed below.

The array in which searching is to be performed is:

Let **x = 4** be the element to be searched.

Set two pointers low and high at the lowest and the highest positions respectively.

Find the middle position mid of the array ie. **mid** = **(low + high)/2** and **arr[mid]** = **6.**

If x == arr[mid], then return mid. Else, compare the element to be searched with arr[mid].

If x > arr[mid], compare x with the middle element of the elements on the right side of arr[mid]. This is done by setting low to low = mid + 1.

Else, compare x with the middle element of the elements on the left side of arr[mid]. This is done by setting high to high = mid - 1.

Repeat steps 3 to 6 until low meets high.

x = 4 is found

Example 18: A C program of a Binary Search using the Iterative Method

```
// Binary Search in C
#include <stdio.h>

int binarySearch(int array[], int x, int low, int high) {
  // Repeat until the pointers low and high meet each other
  while (low <= high) {
    int mid = low + (high - low) / 2;

    if (x == array[mid])
      return mid;

    if (x > array[mid])
      low = mid + 1;

    else
      high = mid - 1;
  }

  return -1;
}

int main(void) {
  int array[] = {3, 4, 5, 6, 7, 8, 9};
  int n = sizeof(array) / sizeof(array[0]);
  int x = 4;
  int result = binarySearch(array, x, 0, n - 1);
  if (result == -1)
    printf("Not found");
  else
    printf("Element is found at index %d", result);
  return 0;
}
```

- **Function Declaration: int binarySearch(int array[], int x, int low, int high):** This declares the binary search function. It takes four parameters:
- **array[]:** The array in which to search.
- **x:** The target value to search for.
- **low:** The starting index of the search range.
- **high:** The ending index of the search range.

The function returns the index of x if found, otherwise -1.

- **While Loop: while (low <= high):** The loop continues as long as low is less than or equal to high, meaning there are still elements to check.
- **Calculate Midpoint: int mid = low + (high - low) / 2;:** Calculates the midpoint index of the current search range to avoid potential overflow by using (low + high) / 2.

- **Check Midpoint: if (x == array[mid]) return mid;:** If the element at the midpoint is equal to x, the function returns mid.
- **Adjust Search Range: if (x > array[mid]) low = mid + 1;:** If x is greater than the element at mid, update low to mid + 1, narrowing the search to the right half.
- else high = mid - 1;: If x is less than the element at mid, update high to mid - 1, narrowing the search to the left half.
- **Return -1: return -1;:** If the loop completes without finding x, the function returns -1, indicating that x is not in the array.

Example 20: A C program of a Binary Search using the Recursion Method

```
// Binary Search in C
#include <stdio.h>
int binarySearch(int array[], int x, int low, int high) {
  if (high >= low) {
    int mid = low + (high - low) / 2;

    // If found at mid, then return it
    if (x == array[mid])
      return mid;

    // Search the right half
    if (x > array[mid])
      return binarySearch(array, x, mid + 1, high);

    // Search the left half
    return binarySearch(array, x, low, mid - 1);
  }

  return -1;
}

int main(void) {
  int array[] = {3, 4, 5, 6, 7, 8, 9};
  int n = sizeof(array) / sizeof(array[0]);
  int x = 4;
  int result = binarySearch(array, x, 0, n - 1);
  if (result == -1)
    printf("Not found");
  else
    printf("Element is found at index %d", result);
}
```

Explanation

- **Function Declaration: int binarySearch(int array[], int x, int low, int high):** Defines a function named binarySearch that takes an array of integers array[], the integer x to search for, and two integers low and high representing the current search bounds. The function returns the index of x if found, otherwise -1.

- **While Loop: while (low <= high):** The loop continues as long as low is less than or equal to high, meaning there are still elements to consider.
- **Calculate Midpoint: int mid = low + (high - low) / 2;:** Computes the midpoint index of the current search range. This avoids potential overflow that could occur with (low + high) / 2.
- **Check If x is at Midpoint: if (x == array[mid]) return mid;:** If the element at the midpoint is x, the function returns mid.
- **Adjust Search Range: if (x > array[mid]) low = mid + 1;:** If x is greater than the element at mid, adjust the search range to the upper half by setting low to mid + 1.
- else high = mid - 1;: Otherwise, adjust the search range to the lower half by setting high to mid - 1.
- **Element Not Found: return -1;:** If the loop exits without finding x, the function returns -1.

This binary search implementation efficiently finds the index of the element x in a sorted array by repeatedly dividing the search interval in half. It ensures that only relevant sections of the array are checked, significantly reducing the number of comparisons needed.

7.1.3. Sentinel Search Algorithm

Sentinel Linear Search is a variation of linear search that adds a sentinel value at the end of the array. The sentinel value is chosen to be the target element, ensuring that the search loop will always terminate. This can improve performance in certain scenarios, especially when the target element is likely to be near the end of the array. The steps of such an algorithm includes;

- **Add Sentinel:** Append the target element to the end of the array as a sentinel.
- **Linear Search:** Perform a linear search on the array, comparing each element with the target element.
- **Termination:** The search loop will terminate when either the target element is found or the sentinel is reached.

Example 21: A C programs that demonstrate how a sentinel search is performed

```
#include <stdio.h>
int sentinelSearch(int arr[], int n, int x) {
    int last = arr[n - 1]; // Store the last element
    arr[n - 1] = x; // Add the sentinel value

    int i = 0;
    while (arr[i] != x) {
        i++;
    }

    if (i < n - 1) {
        return i; // Element found
    } else {
        return -1; // Element not found
    }
}
```

```
int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int n = sizeof(arr) / sizeof(arr[0]);
    int x = 30;

    int result = sentinelSearch(arr, n, x);

    if (result == -1) {
        printf("Element not found\n");
    } else {
        printf("Element found at index %d\n", result);
    }

    return 0;
}
```

Explanation

- The sentinelSearch function takes the array, its size, and the target element as input.
- It stores the last element of the array in a temporary variable last and replaces the last element with the target element (sentinel).
- The while loop iterates through the array until it finds the target element or reaches the sentinel.
- If the target element is found before reaching the sentinel, the function returns its index. Otherwise, it returns -1 to indicate that the element is not present.

Advantages of Sentinel Search

- **Guaranteed termination:** The search loop will always terminate, even if the target element is not present.
- **Can be slightly faster:** In some cases, sentinel search can be slightly faster than traditional linear search, especially when the target element is near the end of the array.

Disadvantages of Sentinel Search

- **Requires extra space:** The sentinel value needs to be stored in an extra element of the array.
- **May not be significantly faster:** The improvement in performance is often marginal and may not justify the extra space requirement.

Question 2: When can one use the Sentinel Search?

- When you need to ensure that the search loop will always terminate, even if the target element is not present.
- When you expect the target element to be near the end of the array.

8. MULTIDIMENSIONAL ARRAY

Multidimensional arrays are data structures that can store elements arranged in multiple dimensions, creating a grid-like structure. They are useful for representing data that has multiple characteristics or can be organized in a tabular format. The most common types of Multidimensional Arrays include;

i. **2D Arrays (Matrices):** A 2D array, also known as a matrix, is a collection of elements arranged in rows and columns. It's a multidimensional data structure that can be visualized as a table or grid. In other words, it represents data in a tabular format with rows and columns. They are mostly used for;
   - ❖ Mathematical operations (e.g., matrix multiplication, addition)
   - ❖ Image processing
   - ❖ Game development

Example 22: A C program that portrays how a 2D Array is used

```
#include <stdio.h>
int main() {
    int matrix[3][4] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12}
    };

    // Accessing elements
    printf("Element at row 1, column 2: %d\n", matrix[0][1]); // Output: 2
    printf("Element at row 2, column 3: %d\n", matrix[1][2]); // Output: 7

    // Modifying elements
    matrix[2][0] = 100;

    // Printing the entire matrix
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 4; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

This code demonstrates the use of a 2D array in C. The matrix array is initialized with values, and elements are accessed and modified using appropriate indices.

Exercise 6: From the matrix below making use of a For loop, access the element at row 1, column 2 displaying the entire matrix

```
int matrix[3][4] = {
      {1, 2, 3, 4},
      {5, 6, 7, 8},
      {9, 10, 11, 12}
   };
```

ii. **3D Arrays:** A 3D array is a multidimensional data structure that can be visualized as a cube. It has three dimensions: ***rows, columns, and layers***. In other words, it represents data in a cube-like structure with rows, columns, and layers. It is mostly used in;

- ❖ Scientific simulations
- ❖ Image processing
- ❖ Game development

Example 23: A C program that portrays how a 3D Array is used

```
#include <stdio.h>
int main() {
   int cube[2][3][4] = {
      {{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}},
      {{13, 14, 15, 16}, {17, 18, 19, 20}, {21, 22, 23, 24}}
   };
   // Accessing elements
   printf("Element at row 1, column 2, layer 0: %d\n", cube[0][1][2]);

   // Iterating through the cube
   for (int i = 0; i < 2; i++) {
      for (int j = 0; j < 3; j++) {
         for (int k = 0; k < 4; k++) {
            printf("%d ", cube[i][j][k]);
         }
         printf("\n");
      }
      printf("\n");
   }

   return 0;
}
```

Note: Layers in a 3D array are like individual "slices" or "sheets" of the array. Imagine a cube where each layer is a 2D matrix. The layers are stacked on top of each other, and you can access elements within a layer using row and column indices, just like in a 2D array.

This code demonstrates the creation, access, and iteration of a 3D array in C. The cube array is initialized with 24 elements arranged in 2 layers, 3 rows, and 4 columns. The code then accesses the element at row 1, column 2, layer 0 (which is 7) and iterates through all elements of the cube to print them.

Exercise 7: Making use of a **For** loop, display all the values from this 3D array

```
int test[2][3][4] = {
      {
         {3, 4, 2, 3},
         {0, -3, 9, 11},
         {23, 12, 23, 2}
      },
      {
         {13, 4, 56, 3},
         {5, 9, 3, 5},
         {3, 1, 4, 9}
      }
```

9. Procedure and Functions

In the context of algorithms, procedures are a type of **subroutine or subprogram** designed to perform a specific task within a larger program. They help in breaking down complex problems into smaller, more manageable parts.

In much more simpler terms, a procedure is a sequence of instructions that performs a specific task. Unlike **functions**, procedures **do not return a value, with the purpose of** encapsulating a set of operations that can be reused throughout a program. This modular approach makes the code easier to understand, maintained, and debugged. A typical procedure includes:

- **Name:** Identifies the procedure.
- **Parameters:** Inputs that the procedure uses to perform its task.
- **Body:** The block of code that defines the operations to be performed.
- **End:** Marks the end of the procedure.

Example 24: Here's a simple example in pseudocode

```
Procedure displayMessage(message)
   Print message
End Procedure

// Calling the procedure
displayMessage("Hello, World!")
```

Procedures are called or invoked from other parts of the program. This invocation can happen multiple times, allowing for code reuse and better organization.

Example 25: A simple procedure to swap two numbers

```
#include <stdio.h>
// Procedure to swap two numbers
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int x = 10, y = 20;
    printf("Before swap: x = %d, y = %d\n", x, y);

    // Calling the swap procedure
    swap(&x, &y);

    printf("After swap: x = %d, y = %d\n", x, y);
    return 0;
}
```

- **Header File:** We use #include **<stdio.h>** for standard input and output functions in C.
- **Procedure Definition:** The swap function now takes pointers to integers **(int *a, int *b)** instead of references. Inside the function, we dereference the pointers to access and swap the values.
- **Procedure Call:** In the main function, we pass the addresses of **x** and **y** to the swap function using the **&** operator.
- **Output:** We use the **printf()** for printing the values before and after the swap.

Exercise 8: Write a C program that demonstrates the concept of procedures by calculating the factorial of a number.

9.1. Functions

Functions are fundamental building blocks that help organize and manage code, but in a more detailed manner, a function is a block of code designed to perform a specific task. It is executed when it is called from another part of the program. Functions help in breaking down complex problems into smaller, manageable pieces, making the code more modular and reusable.

9.1.1. Types of Functions

In C, functions can be broadly categorized into two types: **Standard Library Functions** and **User-Defined Functions.** Let's explore each type along with examples.

i. Standard Library Functions

These are built-in functions provided by C's standard library. They perform common tasks and are defined in various header files. Examples include **printf(), scanf(), strlen(), and sqrt().**

Example 26: Using the **printf() and scanf()**

```
#include <stdio.h>
int main() {
    int num;
    printf("Enter an integer: ");
    scanf("%d", &num);
    printf("You entered: %d\n", num);
    return 0;
}
```

Exercise 9: Write a program that calculates the length of a string entered by the user using the **strlen()** function from the **<string.h>** library.

Exercise 10: Write a program calculates the square root of a number entered by the user using the **sqrt()** function from the **<math.h> library.**

ii. User-Defined Functions

These are functions created by the programmer to perform specific tasks. User-defined functions can be further classified based on their parameters and return values:

a. Function with No Arguments and No Return Value

This type of function does not take any parameters and does not return a value.

Example 27: A program with no parameter and no return value

```
#include <stdio.h>
// Function declaration
void greet();

int main() {
    greet(); // Function call
    return 0;
}

// Function definition
void greet() {
    printf("Hello, World!\n");
}
```

b. Function with No Arguments but Returns a Value

This type of function does not take any parameters but returns a value.

Example 28: A program with no arguments but with a return value

```
#include <stdio.h>
// Function declaration
int getNumber();
```

```
int main() {
    int num = getNumber(); // Function call
    printf("Number: %d\n", num);
    return 0;
}

// Function definition
int getNumber() {
    return 42;
}
```

c. Function with Arguments but No Return Value

This type of function takes parameters but does not return a value.

Example 29: A program with arguments but with no return value

```
#include <stdio.h>
// Function declaration
void printSum(int a, int b);

int main() {
    printSum(5, 10); // Function call
    return 0;
}
// Function definition
void printSum(int a, int b) {
    int sum = a + b;
    printf("Sum: %d\n", sum);
}
```

d. Function with Arguments and Returns a Value

This type of function takes parameters and returns a value.

Example 30: A program with arguments and with return values

```
#include <stdio.h>
// Function declaration
int add(int a, int b);

int main() {
    int result = add(5, 10); // Function call
    printf("Result: %d\n", result);
    return 0;
}

// Function definition
int add(int a, int b) {
```

```
    return a + b;
}
```

In Conclusion

Standard Library Functions are built-in functions provided by C's standard library. Whereas, User-Defined Functions are created by the programmer, which can be:

- With no arguments and no return value.
- With no arguments but returns a value.
- With arguments but no return value.
- With arguments and returns a value.

These classifications help in organizing code, making it modular, reusable, and easier to maintain.

10.ALGORITHM APPROACHES

Algorithm approaches, also known as algorithm design techniques, are strategies used to develop efficient algorithms for solving computational problems. The most common algorithm approaches include**; Brute Force, Divide and Conquer, Dynamic Programming, Greedy Algorithms and more.**

10.1. DIVIDE-AND-CONQUER ALGORITHM

The divide-and-conquer algorithm is a problem-solving technique that involves breaking down a problem into smaller, simpler subproblems, solving each subproblem recursively, and then combining the solutions to obtain the solution to the original problem.

Key Steps

- **Divide:** Divide the original problem into smaller subproblems of the same type.
- **Conquer:** Recursively solve the subproblems.
- **Combine:** Combine the solutions of the subproblems to obtain the solution to the original problem.

Example 29: Here's a simple C program that demonstrates the divide and conquer algorithm using the Merge Sort technique:

```
#include <stdio.h>
// Function to merge two subarrays

void merge (int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];
    int i = 0, j = 0, k = left;
```

```
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
}

// Function to implement Merge Sort
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

// Function to print an array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int arr[] = {12, 11, 13, 5, 6, 7};
    int arr_size = sizeof(arr) / sizeof(arr[0]);
    printf("Given array is \n");
    printArray(arr, arr_size);
    mergeSort(arr, 0, arr_size - 1);
    printf("\nSorted array is \n");
    printArray(arr, arr_size);
    return 0;
```

```
}
```

This program sorts an array using the **Merge Sort algorithm,** which is a classic example of the divide and conquer approach. The array is divided into **two halves**, **each half is sorted recursively, and then the two sorted halves are merged together.**

Explanation of Merge Sort

The merge sort algorithm works by:

- **Dividing:** Dividing the array into two halves repeatedly until each subarray contains only one element.
- **Conquering:** Recursively sorting each subarray using the same merge sort algorithm.
- **Combining:** Merging the sorted subarrays back together into a single sorted array using the merge function.

N.B: Merge Sort is a popular sorting algorithm that follows the divide and conquer strategy.

Breakdown of the Code

Header Inclusion

#include <stdio.h>: This line includes the standard input/output header file, which provides functions for input and output operations like **printf** and **scanf.**

Merge function

Purpose: Merges two sorted subarrays into a single sorted array.

Parameters

- **arr:** The array to be merged.
- **left:** The starting index of the first subarray.
- **mid:** The ending index of the first subarray and the starting index of the second subarray.
- **right:** The ending index of the second subarray.

Steps

- Creates two temporary arrays L and R to store the elements of the left and right subarrays, respectively.
- Copies the elements of the left subarray into L and the elements of the right subarray into R.
- Compares the elements in L and R and merges them back into the original array **arr** in sorted order.

MergeSort Function

Merge Sort is a divide-and-conquer algorithm used to sort an array or list of elements. It works by recursively dividing the array into smaller subarrays, sorting each subarray, and then merging the sorted subarrays to create a fully sorted array.

Purpose: Recursively sorts the array using the divide-and-conquer approach.

Parameters

- **arr:** The array to be sorted.
- **left:** The starting index of the subarray to be sorted.
- **right:** The ending index of the subarray to be sorted.

Steps

- If left is less than right, it means there are more than one element in the subarray.
- Calculates the middle index mid of the subarray.
- Recursively calls **mergeSort** to sort the left subarray **(from left to mid).**
- Recursively calls **mergeSort** to sort the right subarray **(from mid + 1 to right).**
- Calls the merge function to merge the sorted left and right subarrays.

printArray Function

Purpose: Prints the elements of an array to the console.

Parameters

- **arr:** The array to be printed.
- **size:** The size of the array.

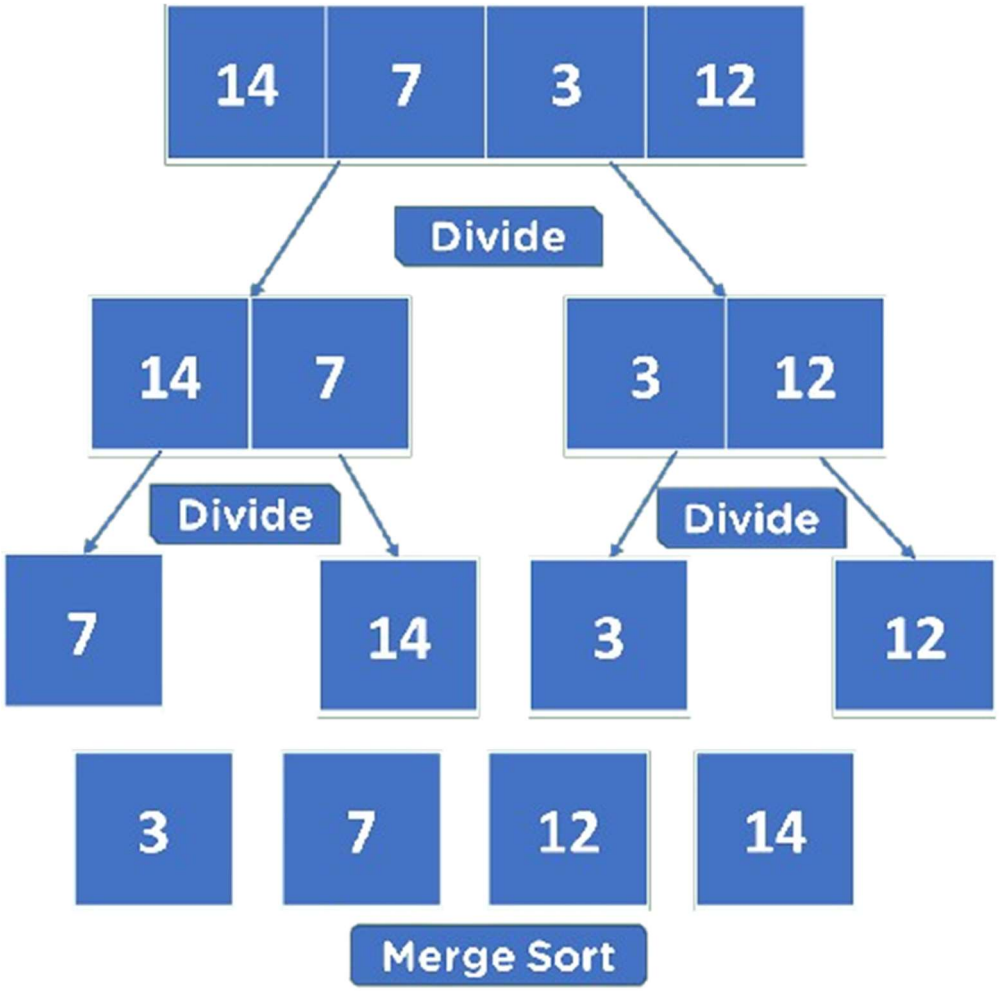
Steps

- Iterates through the array and prints each element.

main Function

- **Array Declaration:** Declares an array named **arr** containing the elements to be sorted.
- **Size Calculation:** Calculates the size of the array.
- **Printing Original Array:** Prints the original unsorted array.
- **Sorting:** Calls the **mergeSort** function to sort the array.
- **Printing Sorted Array:** Prints the sorted array.

Example 29: Here's a diagrammatical explanation along its C program using the Merge Sort Algorithm



Let's dive into the explanation using the diagram and its C code.

Explanation of the Diagram

- **Initial Array: [14, 7, 3, 12]**
- **First Division:** The array is divided into **[14, 7] and [3, 12].**
- **Second Division: [14, 7]** is divided into **[14] and [7]** while, **[3, 12]** is divided into **[3]** and [**12].**
- **Merge: [14] and [7]** are merged to form **[7, 14]** whereas, **[3]** and **[12]** are merged to form **[3, 12].**
- **Final Merge: [7, 14]** and [3, 12] are merged to form the final sorted array **[3, 7, 12, 14].**

```
#include <stdio.h>
// Function to merge two subarrays
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int L[n1], R[n2];

    for (int i = 0; i < n1; i++)
```

```
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
}

// Function to implement merge sort
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

// Function to print the array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

// Main function
```

```
int main() {
    int arr[] = {14, 7, 3, 12};
    int arr_size = sizeof(arr) / sizeof(arr[0]);

    printf("Given array is \n");
    printArray(arr, arr_size);

    mergeSort(arr, 0, arr_size - 1);

    printf("\nSorted array is \n");
    printArray(arr, arr_size);
    return 0;
}
```

Explanation of the Code

Merge Function

- **Splitting Arrays:** L and R arrays hold the left and right halves of the array being merged.
- **Merging Process:** Elements from L and R are compared and placed back into the original array in sorted order.

Merge Sort Function

- **Recursive Division:** The array is recursively split until each subarray has only one element.
- **Merging:** The merge function is called to combine the sorted subarrays.

Print Array Function

- **Printing:** This utility function prints the array elements.

Main Function

- **Initialization:** The main function initializes the array and prints it.
- **Sorting: The mergeSort** function is called to sort the array.
- **Output:** The sorted array is printed.

Exercise 11: Sort the following array using the merge sort algorithm [38, 27, 43, 3, 9, 82, 10, 29].

Divide the array into two halves:

- Left half: ([38, 27, 43, 3])
- Right half: ([9, 82, 10, 29])

Recursively divide each half until each subarray contains only one element:

- Left half: ([38, 27, 43, 3])
- Divide into: ([38, 27]) and ([43, 3])

- Further divide: ([38]), ([27]), ([43]), ([3])
- **Right half: ([9, 82, 10, 29])**
- Divide into: ([9, 82]) and ([10, 29])
- Further divide: ([9]), ([82]), ([10]), ([29])

Merge the subarrays back together in sorted order:

- Merge ([38]) and ([27]) to get ([27, 38])
- Merge ([43]) and ([3]) to get ([3, 43])
- Merge ([27, 38]) and ([3, 43]) to get ([3, 27, 38, 43])
- Merge ([9]) and ([82]) to get ([9, 82])
- Merge ([10]) and ([29]) to get ([10, 29])
- Merge ([9, 82]) and ([10, 29]) to get ([9, 10, 29, 82])

Finally, merge the two sorted halves:

- Merge ([3, 27, 38, 43]) and ([9, 10, 29, 82]) to get the fully sorted array ([3, 9, 10, 27, 29, 38, 43, 82]).

Time Complexity

Merge Sort has a time complexity of O(n log n) in all cases, making it efficient for sorting large datasets.

Advantages of Merge Sort

- **Stable:** Preserves the relative order of elements with equal keys.
- **Efficient:** O(n log n) time complexity.
- **Can be used on linked lists:** Unlike some other sorting algorithms, Merge Sort can be efficiently implemented on linked lists.

Disadvantages of Merge Sort

- **Requires extra space:** Merge Sort requires additional space for the temporary arrays used in the merging process.
- **Can be slower for small arrays:** For very small arrays, simpler algorithms like insertion sort might be faster.

Merge Sort is a versatile and efficient sorting algorithm that is widely used in various applications.

10.2. GREEDY ALGORITHM

A Greedy Algorithm is a problem-solving approach that makes the locally optimal choice at each step with the hope of finding a global optimum. This method is particularly useful for optimization problems where the goal is to find the best solution among many possible ones. In simpler terms, the core idea is to take the best immediate option without worrying about future consequences, leading to a quick and often optimal solution. Some popular examples of algorithms that use such an approach include;

- **Fractional Knapsack Problem:** Select items to maximize the total value within a given weight capacity.
- **Huffman Coding:** Construct optimal prefix codes for characters in a given text.
- **Dijkstra's Algorithm:** Find the shortest path from a source node to all other nodes in a weighted graph.
- **Prim's Algorithm:** Find a minimum spanning tree for a given undirected weighted graph.
- **Kruskal's Algorithm:** Another algorithm for finding a minimum spanning tree.

Key Characteristics of Greedy Algorithms

- **Greedy Choice Property:** If an optimal solution to the problem can be found by choosing the best choice at each step without reconsidering the previous steps once chosen, the problem can be solved using a greedy approach. This property is called greedy choice property whereby a global optimum can be arrived at by selecting the local optimum.
- **Optimal Substructure:** If the optimal overall solution to the problem corresponds to the optimal solution to its subproblems, then the problem can be solved using a greedy approach. This property is called optimal substructure, in which an optimal solution to the problem contains optimal solutions to the subproblems.

Example 30: Coin Change Problem

The Coin Change problem is a classic example of a greedy algorithm. The goal is to make change for a given amount using the smallest number of coins from a given set of denominations.

```
#include <stdio.h>
void findMinCoins(int coins[], int n, int amount) {
    int count[n];
    for (int i = 0; i < n; i++) {
        count[i] = 0;
    }

    for (int i = 0; i < n; i++) {
        while (amount >= coins[i]) {
            amount -= coins[i];
            count[i]++;
        }
    }
    printf("Minimum coins required:\n");
    for (int i = 0; i < n; i++) {
        if (count[i] != 0) {
            printf("%d coin(s) of %d\n", count[i], coins[i]);
        }
    }
}
int main() {
```

```
        int coins[] = {25, 10, 5, 1};
        int n = sizeof(coins) / sizeof(coins[0]);
        int amount = 63;

        findMinCoins(coins, n, amount);
        return 0;
}
```

Explanation

- **Coins Array:** The array coins[] contains the denominations.
- **Amount:** The total amount for which we need to find the minimum number of coins.
- **Greedy Choice:** At each step, the algorithm picks the largest denomination that is less than or equal to the remaining amount.

i. Initialization

- An array count is created to store the number of coins of each denomination used.
- All elements of count are initialized to 0.

ii. Greedy Choice

- The algorithm iterates over the coin denominations, starting from the highest value.
- For each denomination, it repeatedly subtracts the coin's value from the remaining amount until the amount becomes less than the coin's value.
- The number of times a coin is subtracted is incremented in the corresponding count array element.

iii. Result

- The final count array stores the minimum number of coins required for each denomination.

Code Explanation

Header Inclusion:

- **#include <stdio.h>:** Includes the standard input/output library for printing.

findMinCoins Function

- Takes an array of coin denominations coins, the number of coins n, and the target amount amount as input.
- Initializes a count array to store the number of coins used for each denomination.
- Iterates over the coin denominations in descending order:
- While the current amount is greater than or equal to the current coin denomination, subtract the coin's value from the amount and increment the corresponding count.
- Prints the number of coins used for each denomination.

main Function

- Defines an array of coin denominations: 25, 10, 5, 1.

- Sets the target amount to 63.
- Calls the findMinCoins function to find the minimum number of coins required.

Time Complexity: The time complexity of this greedy algorithm is O(n * amount), where n is the number of coin denominations.

Note: While this greedy approach works for many coin denominations, it may not always yield the optimal solution for all cases. For example, if the coin denominations are {1, 3, 4}, and the target amount is 6, the greedy algorithm would use 6 coins of 1, while the optimal solution would use 2 coins (3 + 3).

To ensure an optimal solution for all cases, dynamic programming can be used, which guarantees the minimum number of coins.

Example 31: Activity Selection Problem

The Activity Selection problem involves selecting the maximum number of activities that don't overlap, given their start and end times.

```
#include <stdio.h>
void printMaxActivities(int start[], int end[], int n) {
    int i, j;

    printf("Selected activities are:\n");

    i = 0;
    printf("Activity %d\n", i);
        for (j = 1; j < n; j++) {
        if (start[j] >= end[i]) {
            printf("Activity %d\n", j);
            i = j;
        }
    }
}

int main() {
    int start[] = {1, 3, 0, 5, 8, 5};
    int end[] = {2, 4, 6, 7, 9, 9};
    int n = sizeof(start) / sizeof(start[0]);

    printMaxActivities(start, end, n);
    return 0;
}
```

Explanation

- **Function declaration: printMaxActivities** takes three parameters: arrays start and end (start and end times of activities), and an integer n (the number of activities).
- **Variable declaration:** Declares i and j as integers for iteration.

- **Print header:** printf("Selected activities are:\n"); prints a header.
- **Initialize first activity:** Starts by selecting the first activity (i = 0) and prints it.
- **Loop through activities:** A for loop iterates from the second activity (j = 1) to the last one (j < n).
- **Check compatibility:** if (start[j] >= end[i]) checks if the start time of activity j is greater than or equal to the end time of the last selected activity i.
- **Select activity:** If true, prints Activity %d\n, and updates i to the current activity (i = j).
- **Main function:** Entry point of the program.
- **Initialize arrays:** start and end arrays represent the start and end times of activities.
- **Calculate number of activities:** int n = **sizeof(start) / sizeof(start[0]);** calculates the number of elements in the start array.
- **Call function: printMaxActivities(start, end, n);** invokes the printMaxActivities function with the arrays and their size.
- **End program:** return 0; indicates successful termination.

Greedy Choice: The algorithm selects the activity that finishes first and then selects the next activity that starts after the current one finishes.

Summary

The code selects the maximum number of non-overlapping activities based on their start and end times. It initializes the first activity and iterates through the remaining activities, selecting those that start after the last selected activity ends.

10.3. DYNAMIC PROGRAMMING

Dynamic Programming (DP) is an optimization technique that solves problems by breaking them down into simpler subproblems and storing the results of these subproblems to avoid redundant computations. It's particularly useful for solving problems with **overlapping subproblems** and **optimal substructure.**

Key Characteristics

- **Memoization**: Storing the results of subproblems to avoid redundant calculations.
- **Recursive Approach:** Dynamic programming often involves recursive functions to solve subproblems.
- **Bottom-up Approach:** In some cases, the dynamic programming solution can be implemented iteratively, starting from the smallest subproblems and working up to the larger ones.

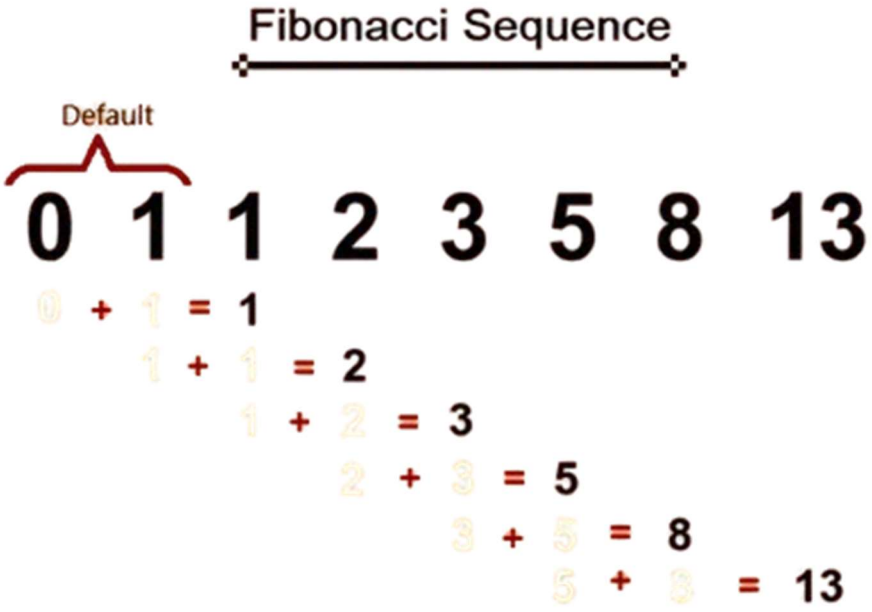
Real-Life Applications of Dynamic Programming

- **Route Optimization:** Think of Google Maps finding the shortest path from point A to B. By using DP, it can efficiently find the shortest route by considering multiple sub routes and storing the shortest paths for reuse.
- **Predictive Text Input:** When you're typing on your smartphone and it suggests words, it uses DP to predict the next word by considering the previous sequences and optimizing the likelihood of the next word.

Example in C programming include:

10.3.1. Fibonacci Sequence

The Fibonacci sequence is a series of numbers where each number is the sum of the two preceding ones, starting from 0 and 1. Where, F(0)=0, F(1)=1. As seen in the image below;



Example 32: Fibonacci sequence that takes the first n numbers

```
#include <stdio.h>
int fibonacci(int n) {
    if (n <= 1) {
        return n;
    } else {
        return fibonacci(n - 1) + fibonacci(n - 2);
    }
}
int main() {
    int n;
    printf("Enter the number of terms: ");
    scanf("%d", &n);

    printf("Fibonacci series:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", fibonacci(i));
    }
    printf("\n");
return 0;
}
```

Explanation

```
#include <stdio.h>
```

This line includes the standard input-output library, allowing you to use functions like **printf and scanf.**

```
int fibonacci(int n) {
    if (n <= 1) {
        return n;
    } else {
        return fibonacci(n - 1) + fibonacci(n - 2);
    }
}
```

This function calculates the Fibonacci number at position n:

- **if (n <= 1) { return n; }:** If n is 0 or 1, it returns n, as the Fibonacci sequence starts with **0, 1.**
- **else { return fibonacci(n - 1) + fibonacci(n - 2); }:** For values of n greater than 1, it recursively calls the function to find the sum of the two preceding numbers.

```
int main() {
    int n;
    printf("Enter the number of terms: ");
    scanf("%d", &n);
```

This block starts the main function:

- **int n;:** Declares an integer variable n to store the number of terms the user wants.
- **printf("Enter the number of terms: ");:** Prompts the user to enter the number of terms.
- **scanf("%d", &n);:** Reads the user input and stores it in n.

```
    printf("Fibonacci series:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", fibonacci(i));
    }
```

This section prints the Fibonacci series:

- **printf("Fibonacci series:\n");:** Prints a header for the Fibonacci series.
- **for (int i = 0; i < n; i++) {:** Initializes a loop from 0 to n-1.
- **printf("%d ", fibonacci(i));:** Calls the fibonacci function for each i from 0 to n-1 and prints the result.

```
    printf("\n");
    return 0;
```

```
}
```

- **printf("\n");:** Prints a newline character after the series.
- **return 0;:** Returns 0, indicating that the program terminated successfully.

This program asks the user for a number of terms and then prints the Fibonacci series up to that number using a recursive function.

Exercise 12: Write an algorithm that checks if a number entered by the user is a fibonacci number or not.

10.4. BRANCH AND BOUND

The Branch and Bound Algorithm is used to solve optimization problems, especially those that are combinatorial in nature, like the traveling salesman problem or knapsack problem. Here's how it works, in essence:

Basic Steps

- **Branching:** This involves dividing the problem into smaller subproblems, effectively creating a **"tree"** of possible solutions. Each node in the tree represents a subproblem.
- **Bounding**: For each subproblem, we calculate a bound that is a best-case **(or worst-case)** estimate of a solution within that subproblem. This helps in determining whether a branch can be discarded.
- **Pruning:** If a subproblem's bound is worse than the current best solution found, we discard that subproblem **(prune the tree).**

Real-life Example

Imagine you're planning a road trip and want to visit several cities with the shortest possible total distance. The Branch and Bound method would start by exploring all possible routes, but would quickly discard **(prune)** any routes that already exceed the shortest route found so far.

Example 32: Let's take a simplified example related to the **knapsack problem: determining the maximum value we can carry without exceeding a weight limit.**

```
#include <stdio.h>
#define MAX_ITEMS 4

//In c programming, a struct (or structure) is a collection of variables (can be of different types) under a single name.
typedef struct {
    int value;
    int weight;
} Item;
int max(int a, int b) {
    return (a > b) ? a : b;
}

int knapsack(int capacity, Item items[], int n) {
    if (n == 0 || capacity == 0)
```

```
    return 0;

  if (items[n-1].weight > capacity)
    return knapsack(capacity, items, n-1);

  return max(items[n-1].value + knapsack(capacity - items[n-1].weight, items, n-1),
        knapsack(capacity, items, n-1));
}

int main() {
  Item items[MAX_ITEMS] = {{60, 10}, {100, 20}, {120, 30}, {80, 40}};
  int capacity = 50;
  printf("Maximum value in knapsack = %d\n", knapsack(capacity, items,
MAX_ITEMS));
  return 0;
}
```

In this example

- Branching happens when we decide whether to include an item or not.
- Bounding ensures that if an item's weight exceeds the capacity, we skip considering it.
- Pruning eliminates branches where the sum of weights already exceeds the capacity.

Explanation

- **Defines a constant MAX_ITEMS with a value of 4,** indicating the number of items.
- Defines a structure **Item** with two integer fields: **value and weight**, representing the value and weight of each item.
- Defines a function **max** that returns the maximum of two integers **a** and **b.**
- Defines the knapsack function, which uses recursion to solve the knapsack problem.
  - **Base case:** If there are no items (n == 0) or the capacity is zero (capacity == 0), it returns 0.
  - If the weight of the n-1th item exceeds the current capacity, it skips this item and calls itself with the remaining items (n-1).

Recursively considers two cases

- Including the n-1th item in the knapsack and adding its value to the result of the remaining capacity **(capacity - items[n-1].weight)** and **items (n-1).**
- Excluding the n-1th item and calling itself with the same capacity and remaining items (n-1).
- Returns the maximum of these two cases using the max function.

The main function

- Initializes an array items with **MAX_ITEMS** elements, each containing value and weight.
- Sets the knapsack capacity to **50.**

- Prints the maximum value that can be obtained by calling the knapsack function with the given capacity, items, and number of items **(MAX_ITEMS).**
- Returns 0, indicating successful program termination.

Purpose

This program demonstrates the use of a recursive approach to solve the knapsack problem, which is a classic optimization problem. It aims to find the maximum value that can be achieved with a given capacity and a set of items, each with a value and weight.

Note: The output is 220 because the program determines the maximum value that can be accommodated in the knapsack without exceeding the given capacity of 50.

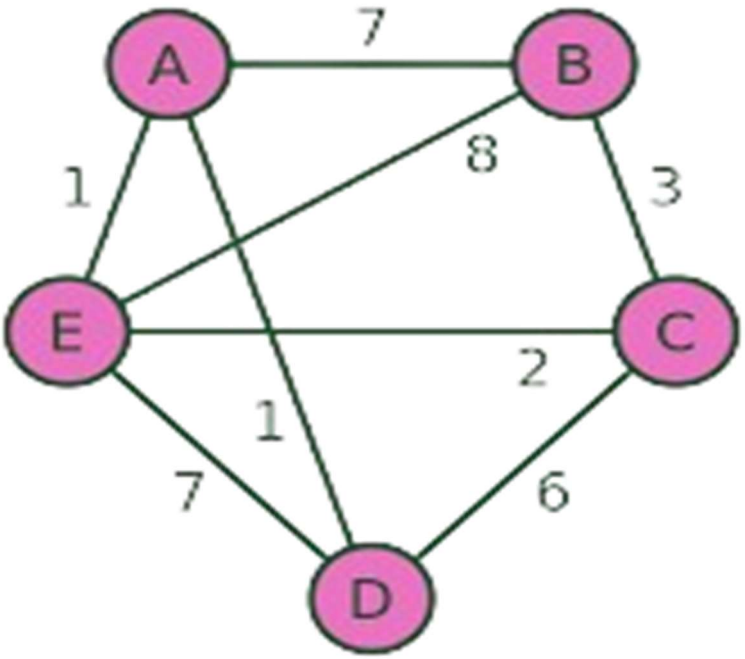
Here's how it works step by step:
There are 4 items with the following values and weights:
Item 1: Value = 60, Weight = 10
Item 2: Value = 100, Weight = 20
Item 3: Value = 120, Weight = 30
Item 4: Value = 80, Weight = 40
The program uses a recursive approach to consider two cases for each item:
Including the item in the knapsack if its weight is within the remaining capacity.
Excluding the item and checking the next one.
For the given capacity of 50, the optimal combination of items is:
Item 2 (Value = 100, Weight = 20)
Item 3 (Value = 120, Weight = 30)
The total value is 100 + 120 = 220, which is why the output is 220.

10.5. BRUTE FORCE

The Brute Force Algorithm, as the name suggests, involves systematically checking all possible solutions to find the one that best solves a given problem. This approach is simple and guarantees finding the correct solution, but it can be highly inefficient, especially for problems with a large number of possibilities.

Example 32: Traveling Salesman Problem (TSP) using the Brute Force Algorithm

Let's consider the TSP where a salesman needs to visit all given cities exactly once and return to the starting city, minimizing the total travel distance.



Explanation

In the brute force approach to TSP:

- Generate all possible permutations of the cities.
- Calculate the total distance for each permutation.
- Select the permutation with the minimum distance.

Below is the C program for the Traveling Salesman Problem using the brute-force algorithm. It will generate all possible routes and calculate the distance for each route to find the shortest one.

```
#include <stdio.h>
#include <limits.h>

#define V 5

int graph[V][V] = {
    {0, 7, 8, INT_MAX, 1},
    {7, 0, 3, 2, INT_MAX},
    {8, 3, 0, 6, 1},
    {INT_MAX, 2, 6, 0, 7},
    {1, INT_MAX, 1, 7, 0}
};
```

```
int minPath = INT_MAX;
int path[V];

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

void copyArray(int *src, int *dest) {
    for (int i = 0; i < V; i++) {
        dest[i] = src[i];
    }
}

void calculatePath(int *route) {
    int currentPathWeight = 0;
    for (int i = 0; i < V - 1; i++) {
        if (graph[route[i]][route[i + 1]] == INT_MAX) {
            return;
        }
        currentPathWeight += graph[route[i]][route[i + 1]];
    }
    if (graph[route[V - 1]][route[0]] == INT_MAX) {
        return;
    }
    currentPathWeight += graph[route[V - 1]][route[0]];

    if (currentPathWeight < minPath) {
        minPath = currentPathWeight;
        copyArray(route, path);
    }
}

void permute(int *route, int l, int r) {
    if (l == r) {
        calculatePath(route);
    } else {
        for (int i = l; i <= r; i++) {
            swap((route + l), (route + i));
            permute(route, l + 1, r);
            swap((route + l), (route + i));
        }
    }
}

int main() {
    int route[V];
    for (int i = 0; i < V; i++) {
```

```
        route[i] = i;
    }

    permute(route, 0, V - 1);

    printf("Minimum path: ");
    for (int i = 0; i < V; i++) {
        printf("%c ", path[i] + 'A');
    }
    printf("%c\n", path[0] + 'A');
    printf("Minimum path weight: %d\n", minPath);

    return 0;
}
```

Code Explanation

i. Graph Representation

- The graph array represents the weighted adjacency matrix of the graph.
- **graph[i][j]** represents the distance between cities i and j.
- A value of **INT_MAX** indicates that there's no direct connection between two cities.

ii. minPath and path Arrays

- **minPath** stores the current minimum cost of a path found so far.
- path stores the current best path.

iii. swap Function:

- Swaps two elements in an array.

iv. copyArray Function

- Copies the contents of one array to another.

v. calculatePath Function

- Calculates the total cost of a given path by summing the distances between adjacent cities.
- If the path is shorter than the current minimum path, it updates minPath and copies the current path to the path array.

vi. permute Function

- Implements a recursive backtracking algorithm to generate all possible permutations of the cities.
- For each permutation, it calls the **calculatePath** function to calculate the total cost and update the **minPath** and path if necessary.

vii. main Function

- Initializes an array route to represent the initial path.
- Calls the permute function to generate all permutations of the cities.

- Prints the minimum path and its cost.

How it Works

viii. Initialization

- The graph matrix is defined to represent the distances between cities.
- The **minPath** is initialized to INT_MAX to store the minimum cost.
- The path array is initialized to store the best path.

ix. Permutation Generation

- The permute function recursively generates all possible permutations of the cities.
- For each permutation, the calculatePath function calculates the total cost of the path.
- If the calculated cost is less than the current minimum cost, it updates the minPath and path variables.

x. Finding the Minimum Path

- After exploring all possible permutations, the minPath and path variables will hold the minimum cost path.

Limitations of Brute Force

While the brute-force approach guarantees finding the optimal solution, it becomes impractical for large problem instances due to its exponential time complexity. For larger problems, more efficient algorithms like dynamic programming or heuristic algorithms are typically used.

11.SEARCHING AND SORTING ALGORITHMS

Searching algorithms are used to find the location of a specific item in a collection of data, examples include **Linear Search algorithm** which scans each element of the array until it finds the target, with a time complexity of **O(n)** and the **Binary Search algorithm** which requires the array to be sorted and proceeds by dividing the array into halves to find the target, with a time complexity of **O(log n).**

Example 33: Linear Search Algorithm

```
#include <stdio.h>
int linearSearch(int arr[], int n, int x) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == x) {
            return i; // Return the index of the element
        }
    }
    return -1; // Return -1 if the element is not found
}

int main() {
    int arr[] = {2, 4, 0, 1, 9};
    int n = sizeof(arr) / sizeof(arr[0]);
    int x = 1;
    int result = linearSearch(arr, n, x);
```

```
    if (result == -1) {
        printf("Element not found in array\n");
    } else {
        printf("Element found at index %d\n", result);
    }
    return 0;
}
```

Example 33: Binary Search Algorithm

```
#include <stdio.h>
int binarySearch(int arr[], int l, int r, int x) {
    while (l <= r) {
        int mid = l + (r - l) / 2;
        if (arr[mid] == x)
            return mid;
        if (arr[mid] < x)
            l = mid + 1;
        else
            r = mid - 1;
    }
    return -1;
}

int main() {
    int arr[] = {2, 3, 4, 10, 40};
    int n = sizeof(arr) / sizeof(arr[0]);
    int x = 10;
    int result = binarySearch(arr, 0, n - 1, x);
    if (result == -1)
        printf("Element is not present in array\n");
    else
        printf("Element is present at index %d\n", result);
    return 0;
}
```

Sorting algorithms are used to rearrange a collection of data into a specific order, usually ascending or descending, examples include the **Bubble Sort** which repeatedly swaps adjacent elements if they are in the wrong order, with a Time Complexity of **O(n^2)** and the **Quick Sort** algorithm which uses a pivot element to partition the array into smaller sub-arrays, which are then sorted with a Time Complexity of **O(n log n)** on average.

Example 34: Bubble Sort Algorithm

```
#include <stdio.h>
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n-1; i++) {
        for (int j = 0; j < n-i-1; j++) {
            if (arr[j] > arr[j+1]) {
```

```
            int temp = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = temp;
        }
    }
  }
}

int main() {
  int arr[] = {64, 34, 25, 12, 22, 11, 90};
  int n = sizeof(arr) / sizeof(arr[0]);
  bubbleSort(arr, n);
  printf("Sorted array: \n");
  for (int i = 0; i < n; i++)
    printf("%d ", arr[i]);
  printf("\n");
  return 0;
}
```

Explanation

- **Function Declaration: void bubbleSort(int arr[], int n):** This declares a function named **bubbleSort** that takes an array of integers **(arr[])** and an integer **(n)** representing the number of elements in the array. The function returns nothing **(void).**
- **Outer Loop:** for **(int i = 0; i < n-1; i++):** This loop runs **n-1** times, where n is the length of the array. It represents the number of passes needed to sort the array.
- **Inner Loop:** for **(int j = 0; j < n-i-1; j++):** This loop goes through the array up to the unsorted portion. With each outer loop iteration, the largest element "**bubbles up**" to its correct position, so the inner loop iterates fewer times.
- **Comparison and Swap: if (arr[j] > arr[j+1]):** If the current element **(arr[j])** is greater than the next element **(arr[j+1]),** they are swapped.
- **Swap Operation:** This code swaps the two elements using a temporary variable **(temp),** but how therefore does it occur?
  These three lines are used to swap two adjacent elements in the array.
  - **Temporary Storage: int temp = arr[j];:** The value of arr[j] (the current element) is stored in a temporary variable called temp.
  - **Assigning the Next Value:** arr[j] = arr[j+1];: The value of arr[j+1] (the next element) is moved into the position of arr[j]. Now, arr[j] has the value of arr[j+1].
  - **Putting the Stored Value:** arr[j+1] = temp;: The value stored in temp (original arr[j]) is assigned to arr[j+1]. This completes the swap.

- **Before the swap:** arr[j] = 34 arr[j+1] = 12
- **Using the steps:** temp = 34 arr[j] = 12 arr[j+1] = 34
- **After the swap:** arr[j] = 12 arr[j+1] = 34
- **Array Declaration: int arr[] = {64, 34, 25, 12, 22, 11, 90};:** An array of integers is defined with the elements to be sorted.

- **Calculate Number of Elements: int n = sizeof(arr)/sizeof(arr[0]);:** This calculates the number of elements in the array. **sizeof(arr)** gives the total size in bytes, and **sizeof(arr[0])** gives the size of one element. Dividing these gives the number of elements.
- **Call Bubble Sort: bubbleSort(arr, n);:** This calls the **bubbleSort** function, passing the array and the number of elements.
- **Print Sorted Array: printf("Sorted array: \n");:** Prints a header for the sorted array.
  - for (int i = 0; i < n; i++) printf("%d ", arr[i]);: This loop prints each element of the sorted array. printf("\n");: Adds a newline after the array is printed.
  - **Return Statement: return 0;:** Indicates that the program executed successfully.

Example 34: Quick Sort Algorithm

```
#include <stdio.h>
void swap(int* a, int* b) {
    int t = *a;
    *a = *b;
    *b = t;
}
int partition (int arr[], int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);
    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int arr[] = {10, 7, 8, 9, 1, 5};
    int n = sizeof(arr) / sizeof(arr[0]);
    quickSort(arr, 0, n-1);
    printf("Sorted array: \n");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
    return 0;
```

```
}
```

11.1. Differences between the Space and Time Complexity in an Algorithm

The Time Complexity of an algorithm measures the amount of time an algorithm takes to complete as a function of the input size. Expressed using the **Big O Notation,** it describes the upper limit of the runtime, focusing on how the runtime of the algorithm grows as the input size increases. Examples include **O(n) for linear time, O(log n) for logarithmic time, O(n^2) for quadratic time.**

Time Complexity Examples

- **O(n):** Indicates that the running time of an algorithm grows linearly with the input size n. That is as the input size increases, the number of operations performed increases proportionally. So, if the input size doubles, the running time roughly doubles, Let's take for instance "***If processing 10 items takes 10 seconds, processing 20 items will take approximately 20 seconds". Examples includes the Linear Search.***

- **O(log n):** Refers to logarithmic time complexity, a highly efficient class of algorithms where the time to complete the task increases logarithmically as the input size n grows. That is to say, the runtime increases slowly even as the input size grows rapidly and doubling the input size only adds a small constant amount to the runtime. Algorithms with **O(log n)** complexity are very efficient, even for large inputs. **Binary Search** is a classic example of an **O(log n)** algorithm. It works on sorted arrays by repeatedly dividing the search interval in half.

- **O(n^2):** It is a notation used to describe the time complexity of an algorithm. It indicates that the algorithm's running time grows **quadratically** with the size of the input data (n). This means that as the **input size doubles**, **the running time roughly quadruples. Examples include the Bubble Sort, Insertion Sort, Selection Sort.**

The Space Complexity measures the amount of memory an algorithm uses relative to the input size, considering the extra memory required by **variables, data structures, and function calls** beyond the input data. And is equally expressed using the **Big O Notation** to describe the upper limit of memory usage. Examples include **O(1)** for **constant space which uses a fixed amount of extra space regardless of input size, whereby an algorithm that uses a fixed amount of extra space regardless of input size. And O(n) an algorithm that uses space proportional to the input size, such as storing a copy of the input array.**

In Summary:

- **Time Complexity:** Focuses on the algorithm's execution time relative to input size.
- **Space Complexity:** Focuses on the algorithm's memory usage relative to input size.
- **Big O Notation:** Standardizes the expression of time and space complexities, helping to evaluate and compare algorithm efficiency.

Note: The Big O Notation is a mathematical notation used to classify algorithms based on their performance and resource usage. It describes the worst-case scenario, providing

an upper bound on time or space complexity. Ignores constant factors and lower-order terms, focusing on the dominant term to express growth rate.

12.DIJKSTRA'S SHORTEST PATH ALGORITHM

This algorithm was created and published by **Dr. Edsger W. Dijkstra,** a brilliant Dutch computer scientist and software engineer. This algorithm is used in **GPS devices** to find the shortest path between the current location and the destination. It has broad applications in industry, especially in domains that require modelling networks.

But before moving any further let's start with a brief introduction to graphs, so one can ask himself what is a graph? Well graphs are data structures used to represent **"connections"** between pairs of elements. These elements are called **nodes.** They represent real-life **objects,** persons, or **entities.** The connections between nodes are called edges. Nodes are represented with **coloured circles** and **edges** are represented with **lines** that connect these circles **(two nodes are connected if there is an edge between them).**

Graphs can be:

- **Undirected:** if for every pair of connected nodes, you can go from one node to the other in both directions.
- **Directed:** if for every pair of connected nodes, you can only go from one node to another in a specific direction. We use arrows instead of simple lines to represent directed edges.

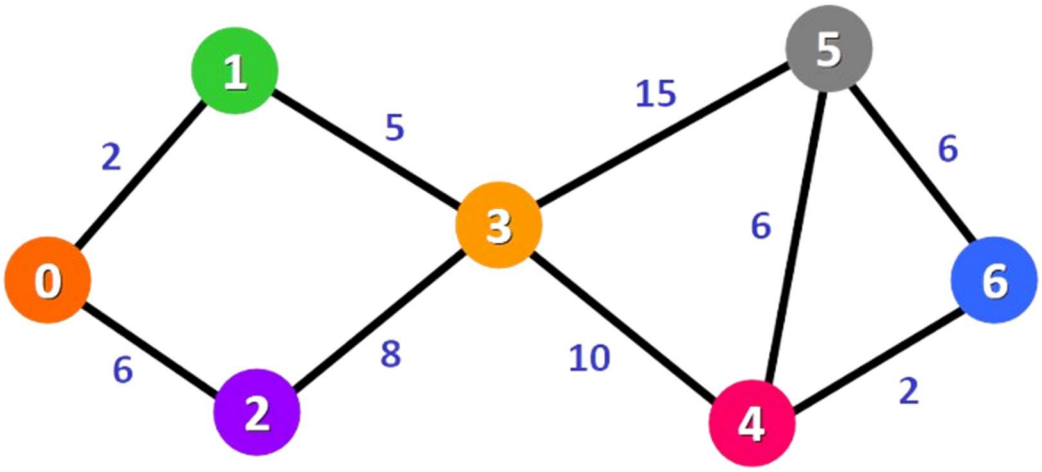
Now in our example below we are working with weighted graphs which is a graph whose edges have a **"weight" or "cost".** The weight of an edge can represent **distance, time,** or anything that models the "connection" between the pair of nodes it connects.

Dijkstra's Algorithm is an algorithm used to find the shortest paths from a source vertex to all other vertices in a graph with non-negative edge weights. It is widely used in network routing protocols, mapping applications, and various optimization problems. But how does it work? Well, all begins at the initialization level whereby;

- **Initialization:** Assign a tentative distance value to every node: set to **zero for the initial node** and **infinity for all other nodes.** Set the initial node as **current** and mark all other nodes as **unvisited.**
- **Visit Neighbouring Nodes:** For the current node, consider all its unvisited neighbours. Calculate their tentative distances through the current node.
- **Update Tentative Distances:** Compare the newly calculated tentative distance to the current assigned value and assign the smaller one.
- **Select Next Node:** Mark the current node as visited and select the unvisited node with the smallest tentative distance as the new current node.
- **Repeat:** Repeat steps 2-4 until all nodes have been visited or the smallest tentative distance among the unvisited nodes is infinity.

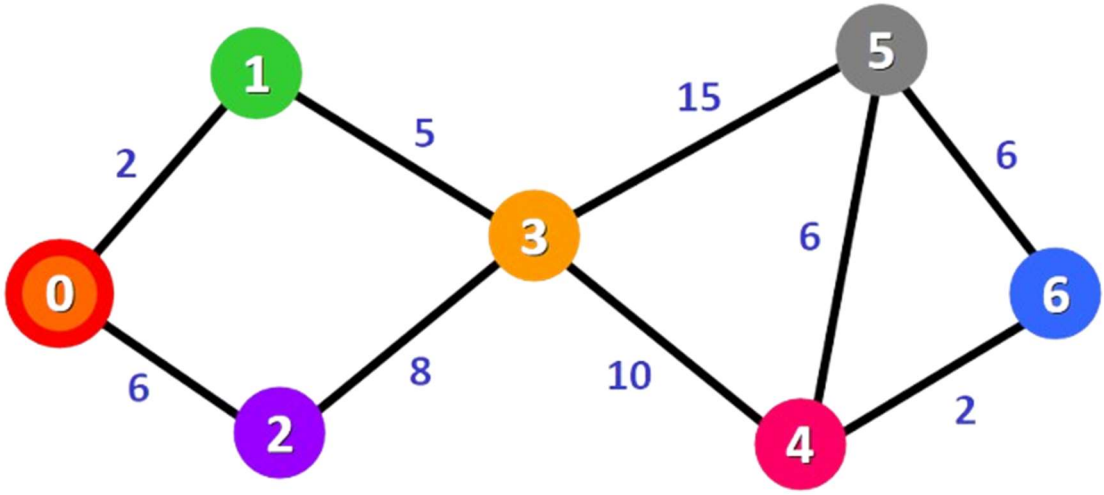
Example of Dijkstra's Algorithm

Now that you know more about this algorithm, let's see how it works behind the scenes with a step-by-step example. So, down below we've got this graph.



The algorithm will generate the shortest path from node 0 to all the other nodes in the graph. For this graph, we will assume that the **weight** of the edges represents the **distance** between two nodes.

We will have the shortest path from **node 0 to node 1,** from **node 0 to node 2**, from **node 0 to node 3, and so on for every node in the graph.** Initially, we have this list of distances and this distance from the source node to all other nodes has not been determined yet, so we use the infinity symbol to represent this initially as represented below.



Distance:

0: 0
1: ∞
2: ∞
3: ∞
4: ∞
5: ∞
6: ∞

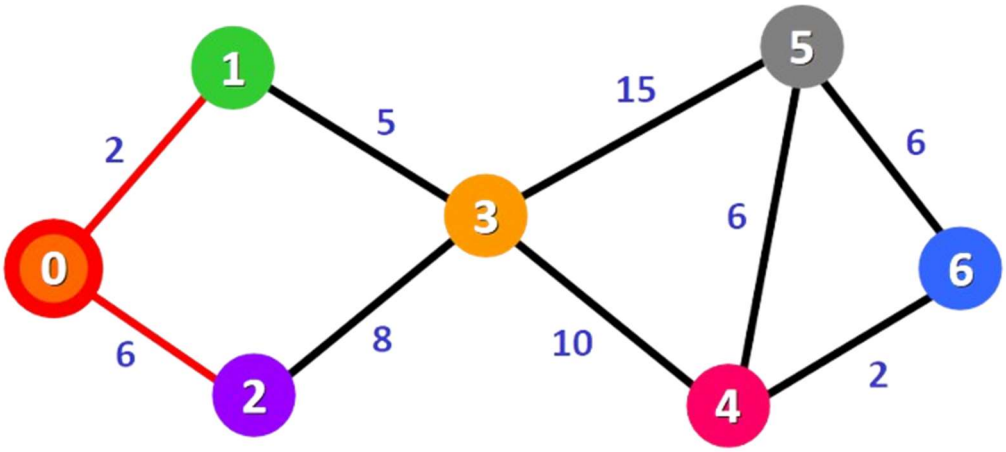
We also have this list to keep track of the nodes that have not been visited yet **(nodes that have not been included in the path):**

Unvisited Nodes: {0, 1, 2, 3, 4, 5, 6}

Before commencing remember, the algorithm is completed once all nodes have been added to the path. Since we are choosing to start at **node 0**, we can mark this node as visited. Equivalently, we cross it off from the list of unvisited nodes and add a red border to the corresponding node in diagram:

Unvisited Nodes: {~~0~~, 1, 2, 3, 4, 5, 6}

Now we need to start checking the distance from **node 0** to its adjacent nodes. As you can see below, these are nodes 1 and 2 as portrayed with the red edges.



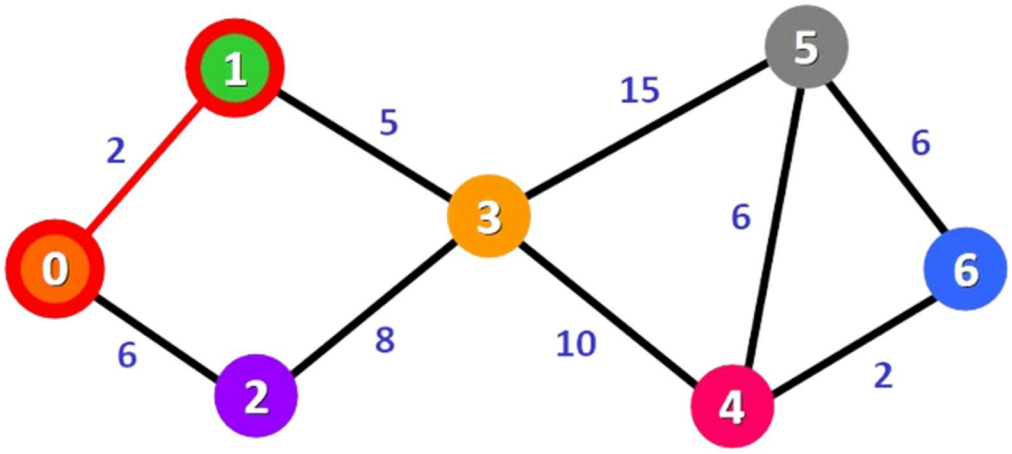
But this doesn't mean that we are immediately adding the two adjacent nodes to the shortest path. Before adding a node to this path, we need to check if we have found the shortest path to reach it. We are simply making an initial examination process to see the options available. Once achieved we need to update the distances from **node 0 to node 1 and node 2** with the weights of the edges that connect them to node 0 (the source node). And these weights are **2 and 6,** respectively:

Distance:

0: 0
1: ~~∞~~ 2
2: ~~∞~~ 6
3: ∞
4: ∞
5: ∞
6: ∞

After updating the distances of the adjacent nodes, we need to select the node that is closest to the source node based on the current known distances, mark it as visited, and add it to the path.

If we check the list of distances, we can see that **node 1** has the shortest distance to the source node **(a distance of 2),** so we add it to the path. In the diagram below, we can represent this with a red edge:



We mark it with a red square in the list of unvisited nodes to represent that it has been **"visited"** and that is it we have found the shortest path to this node.

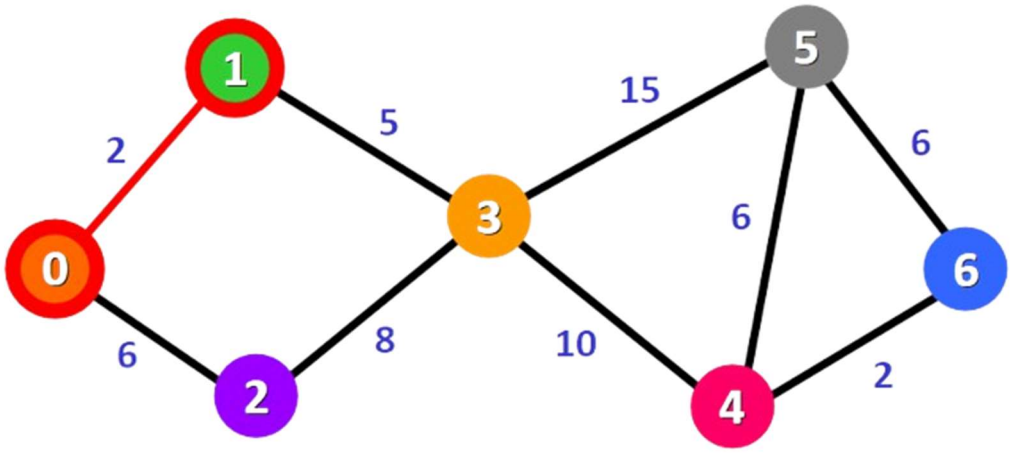
Distance:

0: 0
1: ~~∞~~ 2
2: ~~∞~~ 6
3: ∞
4: ∞
5: ∞
6: ∞

We cross it off from the list of unvisited nodes:

Unvisited Nodes: {~~0~~, ~~1~~, 2, 3, 4, 5, 6}

Now we need to analyse the new adjacent nodes to find the shortest path to reach them. We will only analyse the nodes that are adjacent to the nodes that are already part of the shortest path **(the path marked with red edges). Node 3 and node 2** are both adjacent to nodes that are already in the path because they are directly connected to node 1 and node 0, respectively, as you can see below. These are the nodes that we will analyse in the next step.

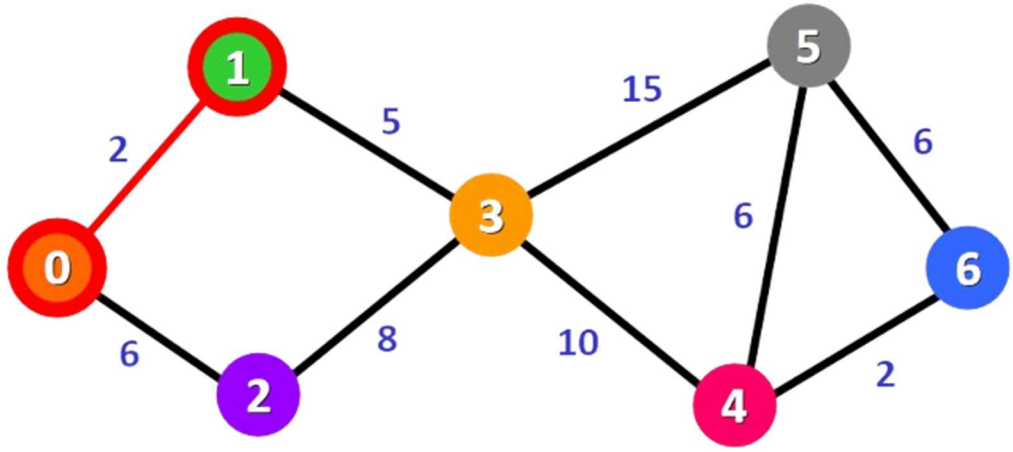


Since we already have the distance from the **source node to node 2** written down in our list, we don't need to update the distance this time. We only need to update the distance from the source node to the new adjacent node **(node 3):**

Distance:

0: 0
1: ~~∞~~ 2
2: ~~∞~~ 6
3: ~~∞~~ 7
4: ∞
5: ∞
6: ∞

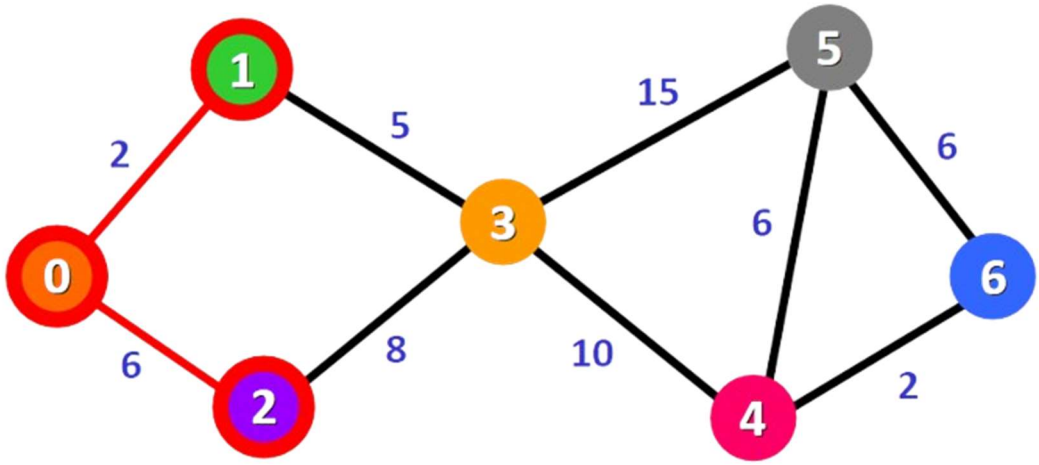
For node 3: the total distance is 7 because we add the weights of the edges that form the path 0 -> 1 -> 3 (2 for the edge 0 -> 1 and 5 for the edge 1 -> 3). Now that we have the distance to the adjacent nodes, we have to choose which node will be added to the path. We must select the unvisited node with the shortest (currently known) distance to the source node. From the list of distances, we can immediately detect that this is node 2 with distance 6:



Distance:

0: 0
1: ~~∞~~ 2
2: ~~∞~~ 6
3: ~~∞~~ 7
4: ∞
5: ∞
6: ∞

We add it to the path graphically with a red border around the node and a red edge:



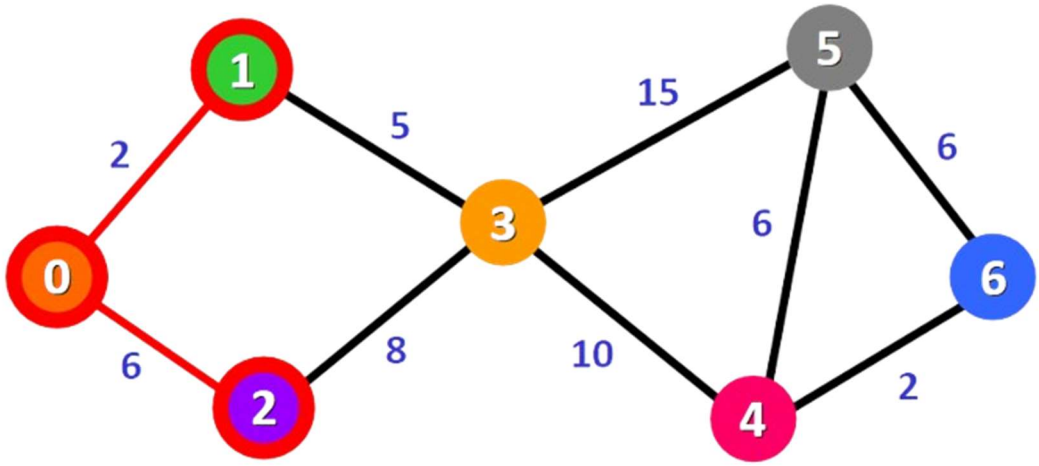
We also mark it as visited by adding a small red square in the list of distances and crossing it off from the list of unvisited nodes:

Distance:

0: 0
1: ~~∞~~ 2
2: ~~∞~~ 6
3: ~~∞~~ 7
4: ∞
5: ∞
6: ∞

Unvisited Nodes: {~~0~~, ~~1~~, ~~2~~, 3, 4, 5, 6}

Now we need to repeat the process to find the shortest path from the source node to the new adjacent node, which is node 3. We can see that we have two possible paths **0 -> 1 -> 3** or **0 -> 2 -> 3.** Let's see how we can decide which one is the shortest path.



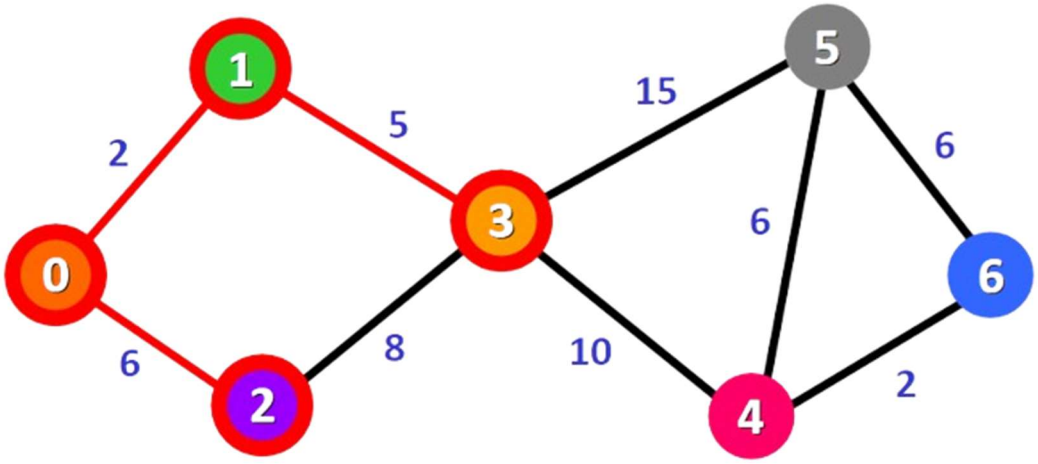
Node 3 already has a distance in the list that was recorded previously (7, see the list below). This distance was the result of a previous step, where we added the weights 5 and 2 of the two edges that we needed to cross to follow the path **0 -> 1 -> 3.**

But now we have another alternative. If we choose to follow the path **0 -> 2 -> 3,** we would need to follow two edges **0 -> 2** and **2 -> 3** with weights 6 and 8, respectively, which represents a total distance of **14.**

Distance:

0: 0
1: ~~∞~~ 2
2: ~~∞~~ 6
3: ~~∞~~ 7 from (5 + 2) vs. 14 from (6 + 8)
4: ∞
5: ∞
6: ∞

Clearly, the first (existing) distance is shorter (7 vs. 14), so we will choose to keep the original path **0 -> 1 -> 3.** We only update the distance if the new path is shorter. Therefore, we add this node to the path using the first alternative: **0 -> 1 -> 3.**



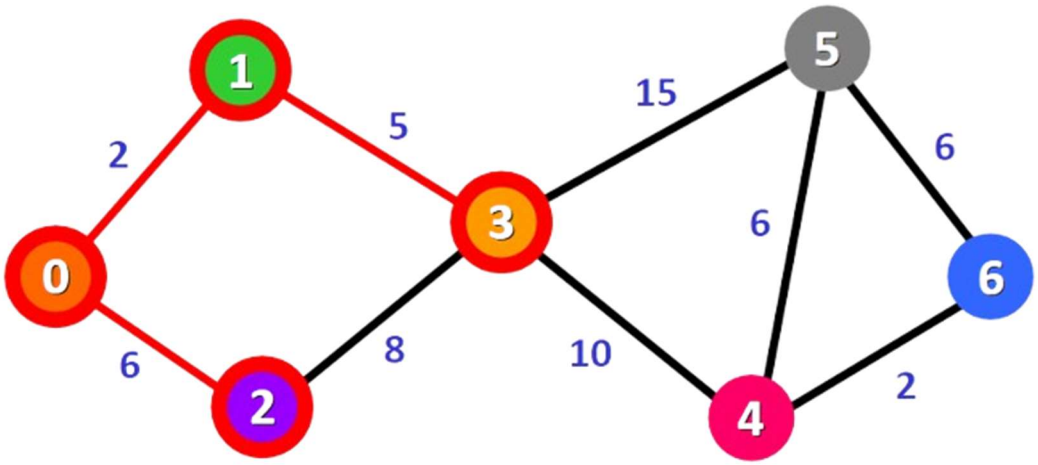
We mark this node as visited and cross it off from the list of unvisited nodes:

Distance:

0: 0
1: ~~∞~~ 2
2: ~~∞~~ 6
3: ~~∞~~ 7
4: ∞
5: ∞
6: ∞

Unvisited Nodes: {~~0~~, ~~1~~, ~~2~~, ~~3~~, 4, 5, 6}

Now we repeat the process again. We need to check the new adjacent nodes that we have not visited so far. This time, these nodes are **node 4 and node 5** since they are adjacent to node 3.



We update the distances of these nodes to the source node, always trying to find a shorter path, if possible:

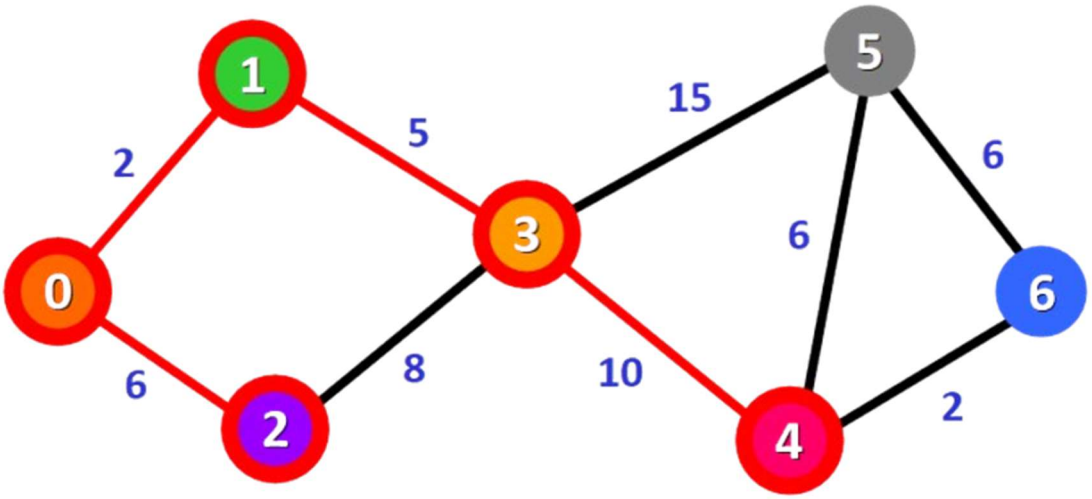
- **For node 4:** the distance is **17** from the path **0 -> 1 -> 3 -> 4.**
- **For node 5:** the distance is **22** from the path **0 -> 1 -> 3 -> 5.**

Note: Notice that we can only consider extending the shortest path (marked in red). We cannot consider paths that will take us through edges that have not been added to the shortest path (for example, we cannot form a path that goes through the edge 2 -> 3).

Distance:

0: 0
1: ~~∞~~ 2 ■
2: ~~∞~~ 6 ■
3: ~~∞~~ 7 ■
4: ~~∞~~ 17 from (2 + 5 + 10)
5: ~~∞~~ 22 from (2 + 5 + 15)
6: ∞

We need to choose which unvisited node will be marked as visited now. In this case, it's **node 4** because it has the shortest distance in the list of distances. We add it graphically in the diagram:



We also mark it as "visited" by adding a small red square in the list:

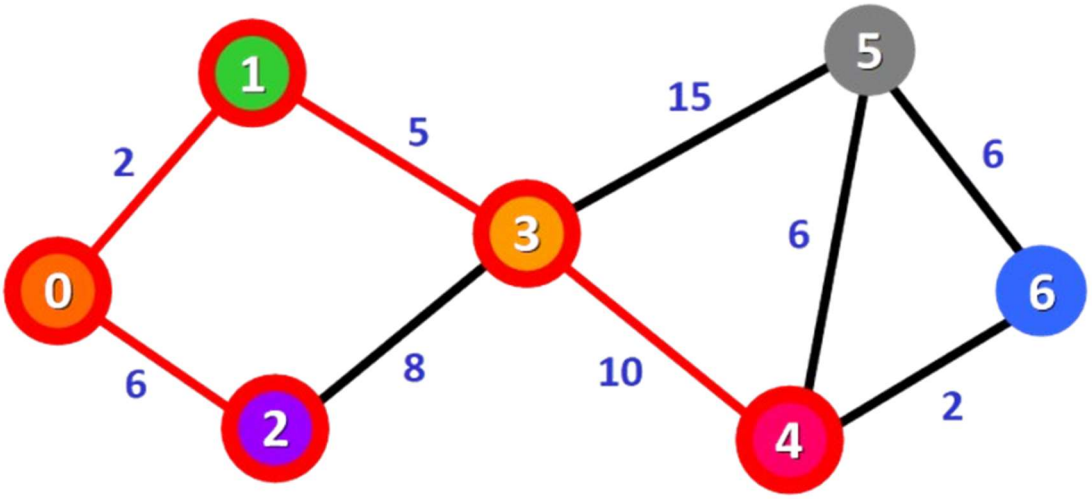
Distance:

0: 0
1: ~~∞~~ 2
2: ~~∞~~ 6
3: ~~∞~~ 7
4: ~~∞~~ 17
5: ~~∞~~ 22
6: ∞

And we cross it off from the list of unvisited nodes:

Unvisited Nodes: {~~0~~, ~~1~~, ~~2~~, ~~3~~, ~~4~~, 5, 6}

And we repeat the process again. We check the adjacent nodes: node 5 and node 6. We need to analyse each possible path that we can follow to reach them from nodes that have already been marked as visited and added to the path.



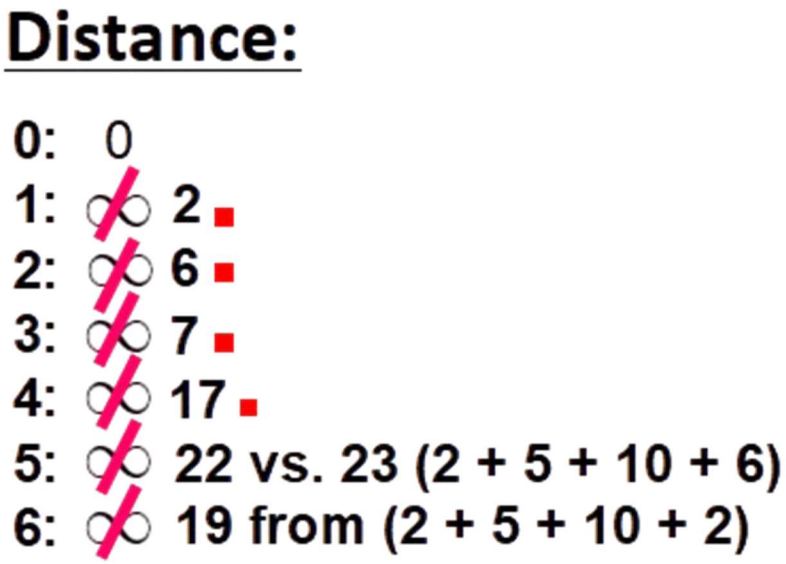
For node 5:

- The first option is to follow the path **0 -> 1 -> 3 -> 5,** which has a distance of 22 from the source node **(2 + 5 + 15).** This distance was already recorded in the list of distances in a previous step.
- The second option would be to follow the path **0 -> 1 -> 3 -> 4 -> 5,** which has a distance of **23** from the source node **(2 + 5 + 10 + 6).**

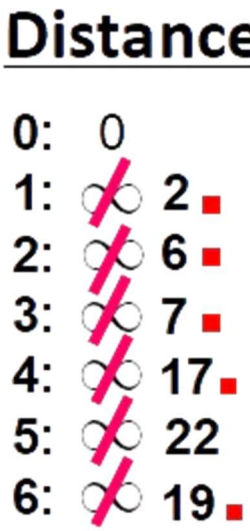
Clearly, the first path is shorter, so we choose it for node 5.

For node 6:

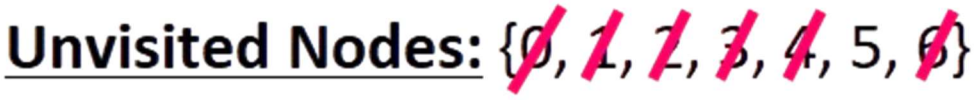
- The path available is 0 -> 1 -> 3 -> 4 -> 6, which has a distance of 19 from the source node (2 + 5 + 10 + 2).



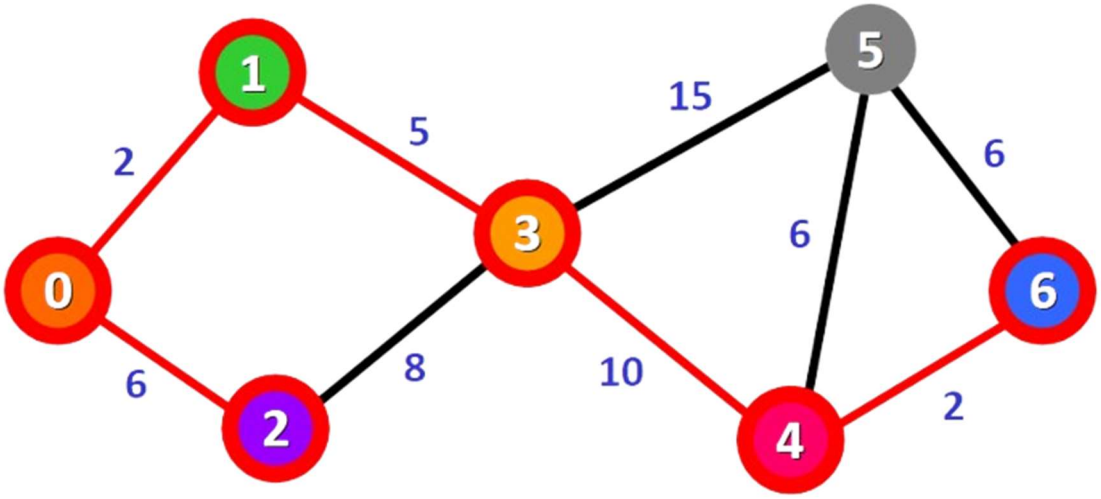
We mark the node with the shortest (currently known) distance as visited. In this case, node 6.



And we cross it off from the list of unvisited nodes:



Now we have this path (marked in red):



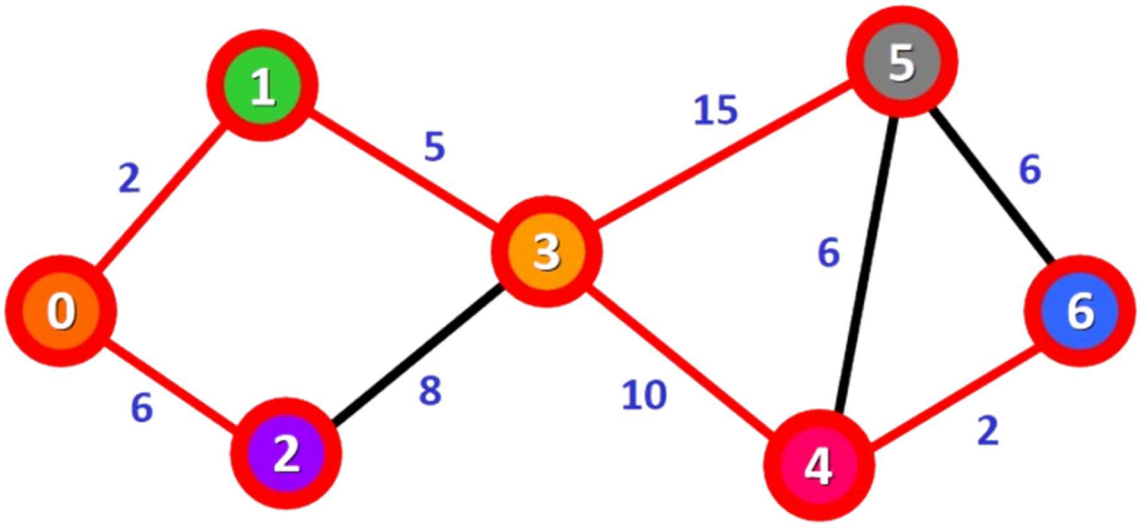
Only one node has not been visited yet, **node 5.** Let's see how we can include it in the path. There are three different paths that we can take to reach node 5 from the nodes that have been added to the path:

- **Option 1: 0 -> 1 -> 3 -> 5** with a distance of **22 (2 + 5 + 15).**
- **Option 2: 0 -> 1 -> 3 -> 4 -> 5** with a distance of **23 (2 + 5 + 10 + 6).**
- **Option 3: 0 -> 1 -> 3 -> 4 -> 6 -> 5** with a distance of **25 (2 + 5 + 10 + 2 + 6).**

Distance:

0: 0
1: ~~∞~~ 2
2: ~~∞~~ 6
3: ~~∞~~ 7
4: ~~∞~~ 17
5: ~~∞~~ 22 from (2 + 5 + 15) vs. 23 from (2 + 5 + 10 + 6) vs. 25 from (2 + 5 + 10 + 2 + 6)
6: ~~∞~~ 19

We select the shortest path: **0 -> 1 -> 3 -> 5** with a distance of **22.**



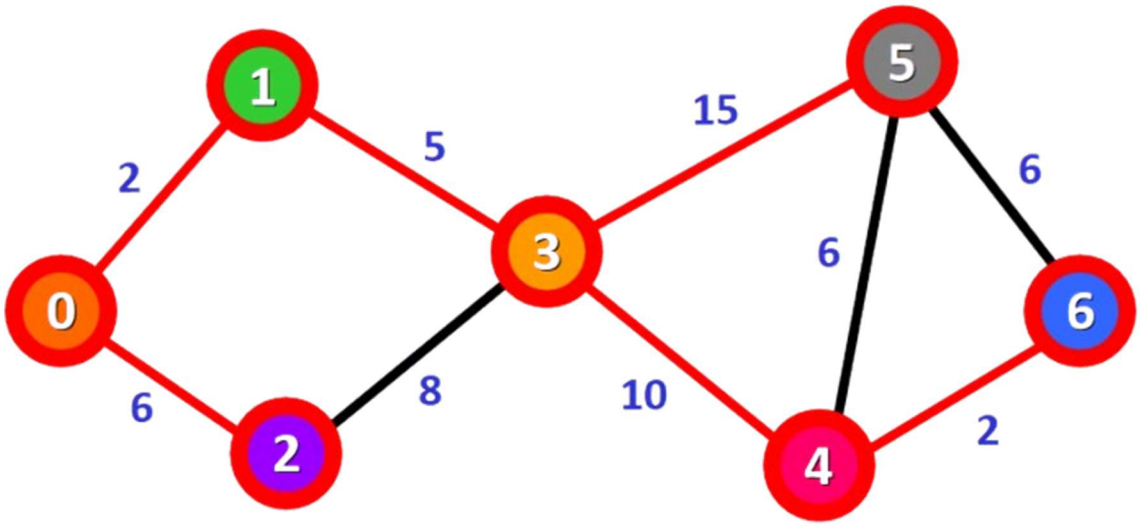
We mark the node as visited and cross it off from the list of unvisited nodes:

Distance:

0: 0
1: ~~∞~~ 2
2: ~~∞~~ 6
3: ~~∞~~ 7
4: ~~∞~~ 17
5: ~~∞~~ 22
6: ~~∞~~ 19

Unvisited Nodes: ~~{0, 1, 2, 3, 4, 5, 6}~~

And voilà! We have the final result with the shortest path from node 0 to each node in the graph.



In the diagram, the red lines mark the edges that belong to the shortest path. You need to follow these edges to follow the shortest path to reach a given node in the graph starting from node 0. For example, if you want to reach node 6 starting from node 0, you just need to follow the red edges and you will be following the shortest path **0 -> 1 -> 3 -> 4 - > 6** automatically.

In Summary

- Graphs are used to model connections between objects, people, or entities. They have two main elements: nodes and edges. Nodes represent objects and edges represent the connections between these objects.
- Dijkstra's Algorithm finds the shortest path between a given node (which is called the "source node") and all other nodes in a graph.
- This algorithm uses the weights of the edges to find the path that minimizes the total distance (weight) between the source node and all other nodes.

Example 35: A C Program for Dijkstra's Algorithm

```
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>

#define V 9

// Function to find the vertex with the minimum distance value, from the set of vertices not
yet included in the shortest path tree
int minDistance(int dist[], bool sptSet[]) {
   int min = INT_MAX, min_index;

   for (int v = 0; v < V; v++)
      if (sptSet[v] == false && dist[v] <= min)
         min = dist[v], min_index = v;

   return min_index;
}
```

```
// Function to print the constructed distance array
void printSolution(int dist[]) {
    printf("Vertex \t Distance from Source\n");
    for (int i = 0; i < V; i++)
        printf("%d \t\t %d\n", i, dist[i]);
}

// Function that implements Dijkstra's single source shortest path algorithm for a graph
represented using adjacency matrix representation
void dijkstra(int graph[V][V], int src) {
    int dist[V]; // The output array. dist[i] will hold the shortest distance from src to i

    bool sptSet[V]; // sptSet[i] will be true if vertex i is included in the shortest path tree

    // Initialize all distances as INFINITE and stpSet[] as false
    for (int i = 0; i < V; i++)
        dist[i] = INT_MAX, sptSet[i] = false;

    // Distance of the source vertex from itself is always 0
    dist[src] = 0;

    // Find the shortest path for all vertices
    for (int count = 0; count < V - 1; count++) {
        // Pick the minimum distance vertex from the set of vertices not yet processed
        int u = minDistance(dist, sptSet);

        // Mark the picked vertex as processed
        sptSet[u] = true;

        // Update dist value of the adjacent vertices of the picked vertex
        for (int v = 0; v < V; v++)

            // Update dist[v] if it's not in sptSet, there is an edge from u to v, and total weight
of path from src to v through u is smaller than current value of dist[v]
            if (!sptSet[v] && graph[u][v] && dist[u] != INT_MAX && dist[u] + graph[u][v]
< dist[v])
                dist[v] = dist[u] + graph[u][v];
    }

    // Print the constructed distance array
    printSolution(dist);
}

int main() {
    int graph[V][V] = {
        {0, 4, 0, 0, 0, 0, 0, 8, 0},
        {4, 0, 8, 0, 0, 0, 0, 11, 0},
        {0, 8, 0, 7, 0, 4, 0, 0, 2},
```

```
        {0, 0, 7, 0, 9, 14, 0, 0, 0},
        {0, 0, 0, 9, 0, 10, 0, 0, 0},
        {0, 0, 4, 14, 10, 0, 2, 0, 0},
        {0, 0, 0, 0, 0, 2, 0, 1, 6},
        {8, 11, 0, 0, 0, 0, 1, 0, 7},
        {0, 0, 2, 0, 0, 0, 6, 7, 0}
    };

    dijkstra(graph, 0);

    return 0;
}
```

Explanation

Header Inclusion

- **#include <stdio.h>:** Includes the standard input/output library for input/output operations.
- **#include <limits.h>:** Includes the limits library for using INT_MAX to represent infinity.
- **#include <stdbool.h>:** Includes the boolean header for using boolean data type.

Macro Definition: #define V 9: Defines a constant V representing the number of vertices in the graph.

minDistance Function: Finds the vertex with the minimum distance value from the set of vertices not yet included in the shortest path tree.

- Iterates through all vertices, checking if they are not included in the sptSet and have a smaller distance than the current minimum.
- Returns the index of the vertex with the minimum distance.

printSolution Function: Prints the calculated shortest distances from the source vertex to all other vertices.

dijkstra Function:

Initialization: Initializes the **dist array** to store the shortest distance from the source to each vertex.

- Initializes the **sptSet array** to keep track of visited vertices.
- Sets the distance of the source vertex to 0.

Main Loop: Iterates V-1 times to find the shortest path for all vertices.

- Finds the vertex with the minimum distance from the set of vertices not yet processed using **minDistance** function.
- Marks the selected vertex as processed.
- Updates the distance values of the adjacent vertices of the selected vertex. If a shorter path is found through the current vertex, the distance is updated.

main Function:

- Defines a graph matrix graph representing the adjacency matrix of the graph.
- Calls the **dijkstra function to** find the shortest paths from the source vertex (vertex 0 in this case) to all other vertices.

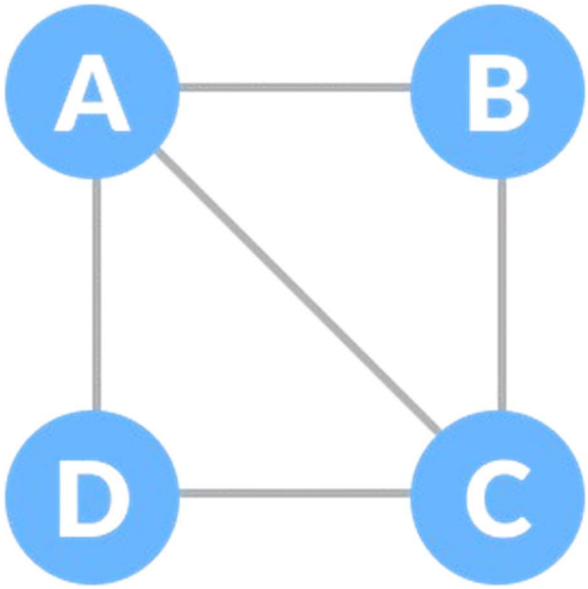
Key Points:

- **Greedy Approach:** Dijkstra's algorithm is a greedy algorithm, always selecting the vertex with the minimum distance from the source.
- **Relaxation:** The process of updating the distance of a vertex if a shorter path is found.
- **Time Complexity:** The time complexity of Dijkstra's algorithm is **O(V^2),** where V is the number of vertices.

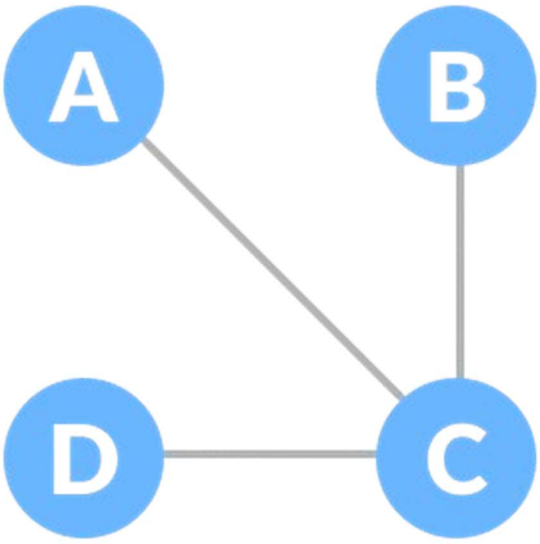
12.1. SPANNING TREE AND MINIMUM SPANNING TREE

Before we learn about spanning trees, we need to understand two graphs: **undirected graphs and connected graphs.**

An undirected graph is a graph in which the edges do not point in any direction as shown below **(ie. the edges are bidirectional).**



A connected graph is a graph in which there is always a path from a vertex to any other vertex as shown below.



Spanning Tree

A spanning tree is a sub-graph of an undirected connected graph, **which includes all the vertices of the graph with a minimum possible number of edges.** If a vertex is missed, then it is not a spanning tree. The edges may or may not have weights assigned to them. The total number of spanning trees with n vertices that can be created from a complete graph is equal to **n(n-2) and is known as Cayley's Formula. It provides the count of different spanning trees that can be formed in a complete graph.** The formula n^{(n-2)} states that for a complete graph with n vertices, the total number of different spanning trees that can be formed is n raised to the power of (n-2). This result is derived using combinatorial methods and has been proved by multiple approaches in graph theory.

For instance, let's say we have **n = 4,** the maximum number of possible spanning trees is equal to $\mathbf{4^{4-2} = 16.}$ Thus, 16 spanning trees can be formed from a complete graph with **4 vertices**.

A Minimum Spanning Tree **(MST)** is a subset of edges from a connected, undirected graph that connects all the vertices together, without any cycles, and with the minimum possible total edge weight. It effectively forms a tree structure that spans all the nodes.

13.TREE AND GRAPH TRAVERSAL TECHNIQUES

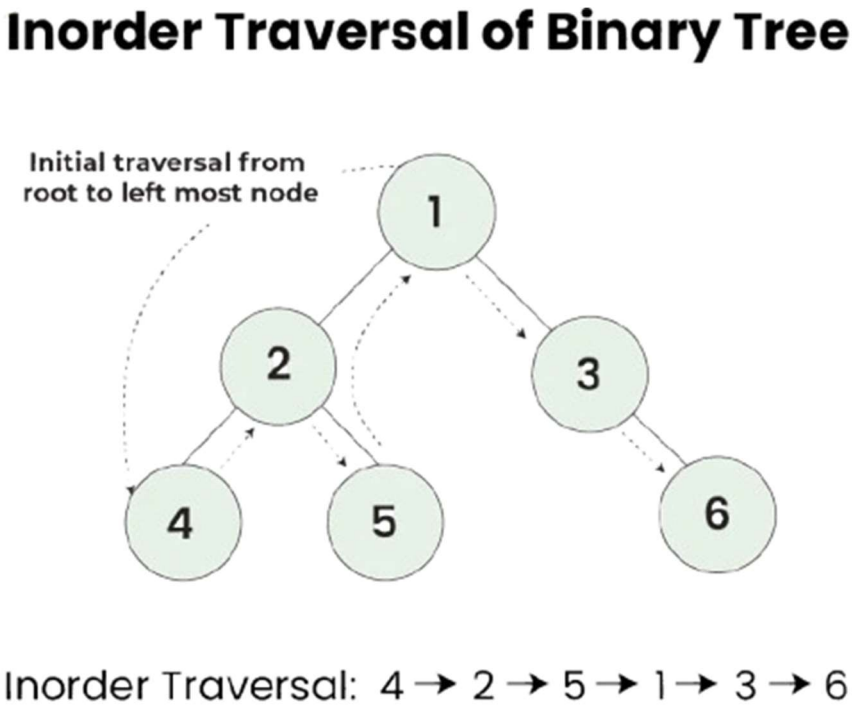
Tree traversal techniques include various ways to visit all the nodes of the tree. Unlike linear data structures **(Array, Linked List, Queues, Stacks, etc)** which have only one logical way to traverse them, trees can be traversed in different ways, therefore we can assume they are used to visit all the nodes of a tree data structure in a specific order. There are three common types of depth-first traversal techniques:

- **Inorder Traversal (Left, Root, Right)**
- **Preorder Traversal (Root, Left, Right)**
- **Postorder Traversal (Left, Right, Root)**

13.1. Inorder Traversal

In Inorder traversal, the nodes are visited in the order: **left subtree, root, right subtree.** This traversal yields nodes in non-decreasing order for a binary search tree.

Example 1: A c program to perform an inorder traversal of the binary tree shown in the diagram below, where the inorder traversal order is: **4, 2, 5, 1, 3, 6.**



Example 35: A C program to demonstrate Inorder Traversal

```
#include <stdio.h>
#include <stdlib.h>

// Definition of a tree node
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}
```

```
// Function to perform inorder traversal
void inorderTraversal(struct Node* node) {
    if (node == NULL) {
        return;
    }
    inorderTraversal(node->left);       // Visit left subtree
    printf("%d ", node->data);          // Visit root
    inorderTraversal(node->right);      // Visit right subtree
}

int main() {
    // Creating the binary tree as shown in the image
    struct Node* root = createNode(1);
    root->left = createNode(2);
    root->right = createNode(3);
    root->left->left = createNode(4);
    root->left->right = createNode(5);
    root->right->right = createNode(6);

    // Performing inorder traversal
    printf("Inorder Traversal: ");
    inorderTraversal(root);
    printf("\n");

    return 0;
}
```

Explanation

- **Node Structure:** Each node contains **data, and pointers to left and right child nodes.**
- **Create Node Function: struct Node* createNode(int data):** Allocates memory for a new node and initializes it with the given data and NULL pointers for left and right children.
- **Inorder Traversal Function: void inorderTraversal(struct Node* node):** Traverses the tree in the order: left subtree, root, right subtree.
- **Base Case:** If the node is NULL, return.
- **Recursive Case:** Traverse the left subtree, print the node's data, then traverse the right subtree.

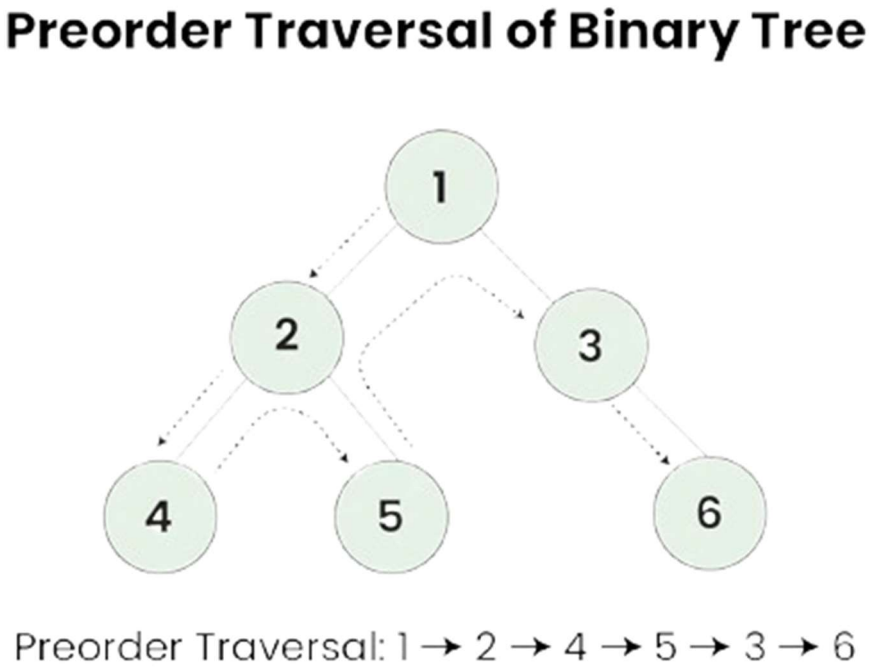
Main Function:

- **Tree Construction:** Creates the binary tree as described.
- **Traversal:** Calls the **inorderTraversal function** to print the nodes in inorder sequence.

13.2. Preorder Traversal

In Preorder traversal, the nodes are visited in the order: **root, left subtree, right subtree.**

Example 36: A C program to perform a preorder traversal of the binary tree shown in the diagram below, where the preorder traversal order is: **1, 2, 4, 5, 3, 6.**




```
#include <stdio.h>
#include <stdlib.h>
// Definition of a tree node
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};
// Function to create a new node
struct Node* newNode(int data) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->data = data;
    node->left = NULL;
    node->right = NULL;
    return node;
}
// Function to perform preorder traversal
void preorderTraversal(struct Node* node) {
    if (node == NULL)
        return;

    // Print the data of the node
    printf("%d ", node->data);

    // Recur on the left subtree
    preorderTraversal(node->left);

    // Recur on the right subtree
    preorderTraversal(node->right);
```

```
}

int main() {
    // Creating the binary tree shown in the image
    struct Node* root = newNode(1);
    root->left = newNode(2);
    root->right = newNode(3);
    root->left->left = newNode(4);
    root->left->right = newNode(5);
    root->right->right = newNode(6);

    // Performing preorder traversal
    printf("Preorder Traversal: ");
    preorderTraversal(root);
    printf("\n");

    return 0;
}
```

Explanation

- **Node Structure:** Each node contains data and pointers to left and right child nodes.
- **Create Node Function: struct Node* newNode(int data):** Allocates memory for a new node and initializes it with the given data and NULL pointers for left and right children.
- **Preorder Traversal Function: void preorderTraversal(struct Node* node):** Traverses the tree in the order: **root, left subtree, right subtree.**
- **Base Case:** If the node is NULL, return.
- **Recursive Case:** Print the node's data, traverse the left subtree, then traverse the right subtree.

Main Function:

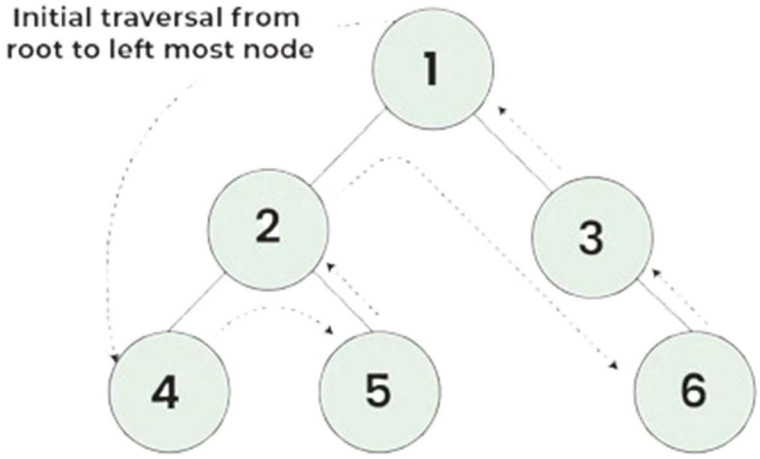
- **Tree Construction:** Creates the binary tree as described.
- **Traversal:** Calls the **preorderTraversal** function to print the nodes in preorder sequence.

Running this program will output the preorder traversal of the tree: **1 2 4 5 3 6.**

13.3. Postorder Traversal

Postorder traversal visits the node in the order: **Left -> Right -> Root.**

Example 36: A c program to perform a postorder traversal of the binary tree shown in the diagram below, where the postorder traversal order is: **4, 5, 2, 6, 3, 1.**



Postorder Traversal: 4 → 5 → 2 → 6 → 3 → 1

```
#include <stdio.h>
#include <stdlib.h>
// Definition of a tree node
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};
// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

// Function to perform postorder traversal
void postorderTraversal(struct Node* node) {
    if (node == NULL)
        return;

    // Traverse the left subtree
    postorderTraversal(node->left);

    // Traverse the right subtree
    postorderTraversal(node->right);

    // Visit the node
```

```
    printf("%d ", node->data);
}

int main() {
    // Creating the tree as shown in the image
    struct Node* root = createNode(1);
    root->left = createNode(2);
    root->right = createNode(3);
    root->left->left = createNode(4);
    root->left->right = createNode(5);
    root->right->right = createNode(6);

    // Performing postorder traversal
    printf("Postorder Traversal: ");
    postorderTraversal(root);
    printf("\n");

    return 0;
}
```

Explanation

- **Node Structure:** Each node contains data and pointers to left and right child nodes.
- **Create Node Function: struct Node* createNode(int data):** Allocates memory for a new node and initializes it with the given data and NULL pointers for left and right children.
- **Postorder Traversal Function: void postorderTraversal(struct Node* node):** Traverses the tree in the order: left subtree, right subtree, root.
- **Base Case:** If the node is NULL, return.
- **Recursive Case:** Traverse the left subtree, traverse the right subtree, then print the node's data.

Main Function:

- **Tree Construction:** Creates the binary tree as described in the image.
- **Traversal:** Calls the **postorderTraversal** function to print the nodes in **postorder** sequence.

Running this program will output the **postorder** traversal of the tree: **4 5 2 6 3 1.**

14.STRING MATCHING

String matching, also known as pattern matching, is a computational technique used to find occurrences of a "pattern" string within a larger "text" string. It's crucial for various applications like text **search engines, DNA sequencing,** and **spam filters.**

14.1. Types of String-Matching Algorithms

14.1.1. Naive String Matching

Compares the pattern to every possible position in the text, with a Time Complexity of O**(n * m),** where **n** is the **length of the text** and **m is the length of the pattern.**

Example 37: Naive String Matching in C

```
#include <stdio.h>
#include <string.h>
// Function to perform naive string matching
void naiveSearch(char* pattern, char* text) {
int m = strlen(pattern);
int n = strlen(text);
for (int i = 0; i <= n - m; i++) {
int j;
for (j = 0; j < m; j++) {
if (text[i + j] != pattern[j])
break;
}
if (j == m) // Pattern found at index i
printf("Pattern found at index %d\n", i);
 }
}
int main() {
char text[] = "AABAACAADAABAAABAA";
char pattern[] = "AABA";
naiveSearch(pattern, text);
return 0;
}
```

Explanation

- **#include <stdio.h>:** Includes the Standard Input Output library which allows the use of functions like printf and scanf.
- **#include <string.h>:** Includes the string handling library which provides functions like strlen.
- **void naiveSearch(char* pattern, char* text):** Defines a function named naiveSearch that takes two parameters: a pattern string and a text string.
- **int m = strlen(pattern);:** Calculates the length of the pattern and stores it in m.
- **int n = strlen(text);:** Calculates the length of the text and stores it in n.
- **Outer Loop: for (int i = 0; i <= n - m; i++)** This loop iterates over each possible starting position in the text where the pattern could fit. It runs from 0 to n - m so that the pattern fits within the bounds of the text.
- **Inner Loop: for (j = 0; j < m; j++)** This loop checks each character of the pattern against the corresponding characters in the text starting at position i.
- **Character Comparison: if (text[i + j] != pattern[j]) break;**

- If any character in the pattern doesn't match the corresponding character in the text, the inner loop breaks.
- **Pattern Found: if (j == m)**
- If the inner loop completes (meaning all characters matched), it means the pattern was found at index i in the text.
- **printf("Pattern found at index %d\n", i);:** Prints the starting index where the pattern was found.

14.1.2. Knuth-Morris-Pratt (KMP) Algorithm

The KMP algorithm is an efficient string-matching algorithm that avoids redundant comparisons by preprocessing the pattern. It uses a partial match table **(also known as the "prefix function")** to skip sections of the text where a mismatch has already been identified. So what we are made to understand here is that it uses **preprocessing** to build a "partial match" table that speeds up the search process, with a Time Complexity of **O(n + m).** But how does it work? Well, it works in two phases which are the;

- **Preprocessing Phase:** Builds a partial match table for the pattern, which indicates the longest prefix of the pattern that is also a suffix up to each point.
- **Matching Phase:** Uses the partial match table to skip unnecessary comparisons in the text.

Real-life scenarios of its application include;

- **Text Editors:** Efficient search and replace functionalities.
- **Spam Filters:** Detecting spam patterns in emails.
- **DNA Sequencing:** Finding gene sequences within larger DNA sequences.

Example 37: Knuth-Morris-Pratt Algorithm in C

```
#include <stdio.h>
#include <string.h>

// Function to build the partial match table (LPS array)
void computeLPSArray(char* pattern, int m, int* lps) {
    int length = 0; // Length of the previous longest prefix suffix
    lps[0] = 0; // LPS[0] is always 0

    int i = 1;
    while (i < m) {
        if (pattern[i] == pattern[length]) {
            length++;
            lps[i] = length;
            i++;
        } else {
            if (length != 0) {
                length = lps[length - 1];
            } else {
                lps[i] = 0;
                i++;
            }
```

```
        }
    }
}

// KMP search algorithm
void KMPsearch(char* text, char* pattern) {
    int n = strlen(text);
    int m = strlen(pattern);

    int lps[m];
    computeLPSArray(pattern, m, lps);

    int i = 0; // Index for text
    int j = 0; // Index for pattern
    while (i < n) {
        if (pattern[j] == text[i]) {
            j++;
            i++;
        }

        if (j == m) {
            printf("Pattern found at index %d\n", i - j);
            j = lps[j - 1];
        } else if (i < n && pattern[j] != text[i]) {
            if (j != 0) {
                j = lps[j - 1];
            } else {
                i++;
            }
        }
    }
}

int main() {
    char text[] = "ABABDABACDABABCABAB";
    char pattern[] = "ABABCABAB";
    KMPsearch(text, pattern);
    return 0;
}
```

Explanation

- **LPS (Longest Prefix Suffix) Array:** Built using the **computeLPSArray** function. Stores the length of the longest proper prefix of the pattern that is also a suffix.
- **KMP Search Function:** Uses the LPS array to efficiently search for the pattern in the text. Skips sections of the text where mismatches are already known, improving performance.

Main Function:

- Defines the text and pattern to search.
- Calls the **KMPsearch** function to perform the search and print the results.

Steps:

- **Compute LPS Array:** Preprocess the pattern to create the LPS array.
- **Search:** Use the LPS array to efficiently match the pattern in the text.

This algorithm drastically reduces the time complexity compared to naive string matching, making it highly useful in applications requiring fast and efficient pattern searching.

14.1.3. Boyer-Moore Algorithm

The Boyer-Moore algorithm is an efficient string-searching algorithm that uses two **heuristics**, the **"bad character rule"** and the **"good suffix rule,"** to skip sections of the text that do not need to be checked making it very efficient for larger texts, with a **Time Complexity of O(n/m).** Therefore, we could ask ourselves how does it work? Well, we begin with the

- **Bad Character Rule:** If a mismatch occurs, the algorithm shifts the pattern so that the bad character in the text aligns with its last occurrence in the pattern.
- **Good Suffix Rule:** If a mismatch occurs, the algorithm shifts the pattern so that a suffix of the matched part of the pattern aligns with another occurrence of that suffix within the pattern.

Real-Life Scenarios

- **Text Editors:** Efficient find and replace functionalities.
- **Search Engines:** Fast search operations within large texts.
- **DNA Sequencing:** Finding gene patterns within large DNA sequences.

Example 38: C Program for Boyer-Moore Algorithm

```
#include <stdio.h>
#include <string.h>

#define NO_OF_CHARS 256

// Function to create the bad character heuristic array
void badCharHeuristic(char* pattern, int size, int badChar[NO_OF_CHARS]) {
    for (int i = 0; i < NO_OF_CHARS; i++)
        badChar[i] = -1;
    for (int i = 0; i < size; i++)
        badChar[(int) pattern[i]] = i;
}

// Boyer-Moore algorithm for string searching
void boyerMooreSearch(char* text, char* pattern) {
    int m = strlen(pattern);
    int n = strlen(text);
```

```
    int badChar[NO_OF_CHARS];
    badCharHeuristic(pattern, m, badChar);

    int s = 0; // s is the shift of the pattern with respect to text
    while (s <= (n - m)) {
        int j = m - 1;

        while (j >= 0 && pattern[j] == text[s + j])
            j--;

        if (j < 0) {
            printf("Pattern found at index %d\n", s);
            s += (s + m < n) ? m - badChar[text[s + m]] : 1;
        } else {
            s += max(1, j - badChar[text[s + j]]);
        }
    }
}

// Utility function to get the maximum of two integers
int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    char text[] = "ABAAABCD";
    char pattern[] = "ABC";
    boyerMooreSearch(text, pattern);
    return 0;
}
```

Explanation

- **Bad Character Heuristic:** void badCharHeuristic(char* pattern, int size, int badChar[NO_OF_CHARS]): Creates an array where each index represents a character, and the value at each index represents the last occurrence of that character in the pattern. Initializes the array to -1 and updates it with the last occurrence of each character in the pattern.
- **Boyer-Moore Search Function: void boyerMooreSearch(char* text, char* pattern):** Uses the bad character heuristic to perform the Boyer-Moore search.
    - Shifts the pattern over the text using both the bad character rule and the good suffix rule to skip unnecessary comparisons.
    - Prints the index of each occurrence of the pattern in the text.

Utility Function: int max(int a, int b): Returns the maximum of two integers, used to determine the shift distance.

Main Function:

- Defines the text and pattern to search.
- Calls the **boyerMooreSearch** function to perform the search and print the results.

Steps:

- **Create Bad Character Heuristic:** Preprocess the pattern to create the bad character array.
- **Search:** Use the bad character heuristic to efficiently search for the pattern in the text.

The Boyer-Moore algorithm's efficiency makes it highly suitable for large-scale text searches and pattern matching.

14.1.4. Rabin-Karp Algorithm

The Rabin-Karp algorithm is a string-searching algorithm that uses hashing to find any one of a set of pattern strings in a text having a Time Complexity on Average **O(n + m).** It's particularly efficient when searching for multiple patterns in a text. How does it work? First through;

- **Hashing:** The algorithm computes a hash value for the pattern and for each substring of the text that is the same length as the pattern.
- **Sliding Window:** The hash values are compared. If the hash values match, a further check is done to confirm the match by comparing the actual characters (to handle hash collisions).

Real life scenarios include;

- **Plagiarism Detection:** Quickly find matches of suspected plagiarized text in large documents.
- **Text Editing:** Implement efficient find-and-replace operations.
- **Bioinformatics:** Search for patterns in DNA sequences.

Example 39: C Program for Rabin-Karp Algorithm

```
#include <stdio.h>
#include <string.h>

#define d 256 // Number of characters in the input alphabet
#define q 101 // A prime number

void rabinKarp(char* pattern, char* text) {
    int m = strlen(pattern);
    int n = strlen(text);
    int p = 0; // Hash value for pattern
    int t = 0; // Hash value for text
    int h = 1;
    int i, j;

    // The value of h would be "pow(d, m-1) % q"
    for (i = 0; i < m - 1; i++)
```

```
    h = (h * d) % q;

  // Calculate the hash value of pattern and first window of text
  for (i = 0; i < m; i++) {
    p = (d * p + pattern[i]) % q;
    t = (d * t + text[i]) % q;
  }

  // Slide the pattern over text one by one
  for (i = 0; i <= n - m; i++) {
    // Check the hash values of current window of text and pattern
    if (p == t) {
      // Check for characters one by one
      for (j = 0; j < m; j++) {
        if (text[i + j] != pattern[j])
          break;
      }
      if (j == m) // if p == t and pattern[0...m-1] = text[i, i+1, ...i+m-1]
        printf("Pattern found at index %d\n", i);
    }

    // Calculate hash value for next window of text
    if (i < n - m) {
      t = (d * (t - text[i] * h) + text[i + m]) % q;

      // We might get negative value of t, converting it to positive
      if (t < 0)
        t = (t + q);
    }
  }
}

int main() {
  char text[] = "IUGET IS AWESOME";
  char pattern[] = "IUGET";
  rabinKarp(pattern, text);
  return 0;
}
```

Explanation

Constants

- **#define d 256:** Defines the number of characters in the input alphabet.
- **#define q 101:** Defines a prime number used for hashing.

Rabin-Karp Function:

- **Hash Calculation:** Computes the initial hash value for the pattern and the first window of the text.

- **Sliding Window:** Slides the pattern over the text and compares hash values.
- **Hash Matching:** If the hash values match, it checks the actual characters to confirm the match.
- **Hash Recalculation:** Updates the hash value for the next window of the text.

Main Function:

- **Text and Pattern:** Defines the text and the pattern to search.
- **Function Call:** Calls the **rabinKarp** function to perform the search and print the results.

Steps:

- **Initial Hash Calculation:** Compute hash values for the pattern and the initial text window.
- **Pattern Sliding:** Slide the pattern over the text and check for matches using hash values.
- **Collision Handling:** Verify matches by comparing actual characters when hash values match.

The Rabin-Karp algorithm's use of hashing makes it efficient for multiple pattern searches and large texts.